



AeroPico FC — picoport2

RP2350 (Raspberry Pi Pico 2) Port — Profesyonel Yazılım Mimarisi İnceleme Raporu

*Sabit Kanatlı İHA Uçuş Kontrolcüsü Yazılımı — Kaynak Kod Denetimi
v0.2.0 → picoport2 sürüm karşılaştırmalı takip incelemesi*

Hazırlayan: Claude (Anthropic) — otomatik statik kod denetimi

İnceleme Tarihi: 1 Temmuz 2026

İncelenen Dal: AeroPico-FC-picoport2 (kaynak zip)

Hedef Donanım: Raspberry Pi Pico 2 — RP2350, Dual-Core ARM Cortex-M33

Toplam Kaynak Kodu: ~2.415 satır C++ (src/, .pio/libdeps hariç)

Önceki Referans: AeroPico_FC_v020_Architecture_Review.pdf (Haziran 2026, proje deposunda mevcut)

Bu belge, projenin /docs klasöründe bulunan Haziran 2026 tarihli v0.2.0 incelemesinin doğrudan devamı olarak; picoport2 dalında yapılan değişiklikleri, çözülen sorunları, kalıcı eksikleri ve RP2350 portu sırasında ortaya çıkan yeni bulguları kod satırı düzeyinde belgeler.

İÇİNDEKİLER

İÇİNDEKİLER	2
BÖLÜM 1 — Yönetici Özeti (Executive Summary)	5
1.1 Genel Değerlendirme	5
1.2 Güçlü Yönler	5
1.3 v0.2.0 → picoport2: Değişim Takip Tablosu	6
1.4 Kritik Eksikler ve Riskler (Güncel Durum)	7
1.5 Profesyonellik Puanı (Genel Özet)	7
1.6 Ticari Potansiyel	8
BÖLÜM 2 — Yazılım Mimarisi	9
2.1 Katmanlı Mimari Analizi	9
2.2 Dosya Organizasyonu ve Modüler Yapı	9
2.3 Modüller Arası Bağımlılıklar	10
2.4 SOLID Prensipleri Analizi	11
2.5 Tasarım Desenleri	11
2.6 Kod Karmaşıklığı	11
2.7 Ölçeklenebilirlik Değerlendirmesi	11
BÖLÜM 3 — Gerçek Zamanlı Sistem Analizi	13
3.1 FreeRTOS Görev Mimarisi	13
3.2 Core Affinity ve Görev Zamanlaması	13
3.3 Mutex ve Senkronizasyon	14
3.4 Lock-Free Ring Buffer (SPSC) Analizi — Kritik Bulgu	15
3.5 PIO State Machine Analizi	15
3.6 Worst Case Execution Time (WCET) Tahmini	16
3.7 Watchdog ve Priority Inversion — Yeniden Değerlendirme	16
BÖLÜM 4 — Sensör Katmanı Analizi	18
4.1 MPU6050 Sürücüsü — Ham I2C + DMA	18
4.2 DMA Pipeline'inin Tamlığı	18
4.3 GY-87 Modülü — İsimlendirme Uyuşmazlığı (Yeni Bulgu)	18
4.4 Blocking I2C — Mag/Baro Okumaları	19
4.5 Sensör Doğrulama Özeti	19
4.6 Kalibrasyon Eksikliği	19
BÖLÜM 5 — Sensor Fusion Analizi	20
5.1 Madgwick Filtresi — 9-DOF update()	20
5.2 6-DOF updateIMU() — Önceki Bulgu Çözüldü	20

5.3 Sıcaklık Kompanzasyonu	20
5.4 Kuaterniyon → Euler Dönüşümü ve Gimbal Lock — Yeni Bulgu	20
5.5 invSqrt() Fonksiyonu	21
5.6 Gürültü ve Drift Değerlendirmesi	21
BÖLÜM 6 — Kontrol Algoritmaları Analizi	22
6.1 PID Kontrolcüsü ve Anti-Windup — İyileştirme Doğrulandı	22
6.2 Cascaded PID Mimarisi	22
6.3 RC Giriş İşleme	22
6.4 Mixer Kod Tekrarı — Değişmedi	23
6.5 Feed-Forward Eksikliği — Değişmedi	23
6.6 Fixed-Wing Kontrol Özellikleri	23
BÖLÜM 7 — Güvenlik Analizi	24
7.1 Failsafe Mekanizması	24
7.2 Watchdog Timer — Kısmen Çözüldü, Yeni Kör Nokta	24
7.3 Stack Overflow Koruması — Çözüldü	24
7.4 Heap Yönetimi — Çözüldü	24
7.5 Arm/Disarm Güvenliği	24
7.6 Brownout / Safe Mode — Değişmedi	25
7.7 Boot Sağlık Raporu — Yeni Kritik Bulgu	25
BÖLÜM 8 — Haberleşme Analizi	26
8.1 MAVLink v2 Implementasyonu	26
8.2 SYS_STATUS Mesajı — Küçük Bulgu	26
8.3 Blackbox Logger	26
8.4 Parametre Sistemi — Devre Dışı ve Bağlantısız	27
BÖLÜM 9 — Pico 2 (RP2350) Donanım Analizi	28
9.1 RP2350 Özelliklerinin Kullanımı	28
9.2 __not_in_flash_func() Kullanımı	28
9.3 Saat (Clock) Yapılandırması	28
9.4 Bellek Haritası (Statik Analizden Tahmini)	29
9.5 GPIO / UART Eşlemesi — Kritik Yeni Bulgu	29
BÖLÜM 10 — PX4 / ArduPilot Karşılaştırması	30
10.1 Özellik Karşılaştırma Tablosu	30
10.2 AeroPico'nun Üstün Olduğu Alanlar	31
10.3 Geliştirilmesi Gereken Alanlar	31
BÖLÜM 11 — Kod Denetimi (Bug Hunt) — Konsolide Liste	32
11.1 Kritik Hatalar (Uçuş Güvenliğini Doğrudan Etkileyebilir)	32

BUG-101 — SBUS RX, GPIO1'de Desteklenmeyen Bir UART'a Bağlanıyor	32
BUG-102 — Watchdog, Core 1 Kilitlenmesini Tespit Edemiyor	32
BUG-103 — Boot Sağlık Raporu Sabit-Doğru Değerler Basıyor	32
11.2 Orta Seviye Sorunlar	32
WARN-101 — RingBuffer SPSC Varsayımı İhlal Ediliyor (Çoklu Tüketici)	32
WARN-102 — asin() Argümanı Sınırlanmıyor (Gimbal Lock Bölgesinde NaN Riski)	32
WARN-103 — ParamManager Mesaj Yönlendiricisine Bağlı Değil	32
WARN-104 — Sensör İsimleri Boot Logunda Yanlış Raporlanıyor	32
WARN-105 — SYS_STATUS Batarya Alanı Anlamsız Değer Taşıyor	33
WARN-003 (taşınan) — Mixer.h Kullanılmıyor	33
WARN-004 (taşınan) — IMU.cpp Kullanılmıyor	33
WARN-001 (taşınan) — _mpu_start_dma_read() İçinde Blocking I2C Yazımı	33
WARN-002 (taşınan) — Wire.h Üzerinden Blocking Mag/Baro Okuması	33
11.3 Düşük Öncelikli Uyarılar	33
BÖLÜM 12 — Geliştirme Yol Haritası	34
12.1 v0.3 — Acil Düzeltmeler (1-2 hafta)	34
12.2 v0.4 — Temizlik ve Sağlama (2-4 hafta)	34
12.3 v0.5 — Özellik Genişletme (1-2 ay)	34
12.4 v1.0 — Kararlı Sürüm (3-5 ay)	34
12.5 v2.0 — Platform Genişlemesi	35
EKLER	36
Ek A — Görev / Zamanlama Özeti	36
Ek B — Risk Matrisi	36
Ek C — Modül Puan Özeti	36
Ek D — Önceliklendirilmiş TODO Listesi	37
Kapanış Notu	37

BÖLÜM 1 — Yönetici Özeti (Executive Summary)

AeroPico FC, Raspberry Pi Pico / Pico 2 (RP2040 / RP2350) üzerinde çalışan, sabit kanatlı İHA'lar için geliştirilmiş açık kaynaklı bir uçuş kontrolcüsü ateşleme yazılımıdır (firmware). İncelenen 'picoport2' dalı, RP2350 hedefine taşınmış (port edilmiş) sürümü temsil etmektedir. Kaynak ağacında (src/) toplam 47 dosya ve yaklaşık 2.415 satır C++ kodu bulunmaktadır; bu rakam, projenin deposunda hâlihazırda bulunan Haziran 2026 tarihli 'v0.2.0 Architecture Review' raporundaki 2.200 satırlık tahminle uyumludur ve projenin picoport2 dalında iskeletin büyük ölçüde korunduğunu, ancak önemli iç değişiklikler geçirdiğini göstermektedir.

Bu rapor, söz konusu önceki incelemenin bağımsız bir tekrarı değil, doğrudan devamıdır: her bulgu, önceki raporun ilgili maddesiyle karşılaştırılmış; 'ÇÖZÜLDÜ', 'KISMEN ÇÖZÜLDÜ', 'DEĞİŞMEDİ' veya 'YENİ' olarak sınıflandırılmıştır. Analiz, projenin README.md, platformio.ini yapılandırması ve tüm src/ altındaki kaynak dosyaları satır satır okunarak yürütülmüştür; hiçbir bulgu varsayıma dayanmamaktadır.

1.1 Genel Değerlendirme

picoport2 dalı, önceki incelemede tespit edilen en kritik üç güvenlik açığından ikisini gerçek anlamda kapatmıştır: watchdog artık etkinleştirilmekte, FreeRTOS stack-overflow ve malloc-failed hook'ları artık tanımlanmaktadır ve heap raporlaması artık donanımdan (rp2040.getFreeHeap()) dinamik olarak okunmaktadır. PID anti-windup, basit clamp yönteminden koşullu entegrasyon (conditional integration) yöntemine yükseltilmiş; MavlinkHandler ↔ FlightManager arasındaki sıkı bağıllık (tight coupling), önerilen tam olarak callback tabanlı bağımlılık enjeksiyonuna (dependency injection) dönüştürülmüş; RC_CHANNELS_OVERRIDE artık uçtan uca çalışır durumdadır. Bunlar, bir sonraki geliştirme turunun somut ve doğrulanabilir ilerlemeler kaydettiğini göstermektedir.

Buna karşılık, bu incelemede RP2350 portuna özgü olduğu değerlendirilen ve önceki raporda yer almayan, uçuşu doğrudan etkileme potansiyeli taşıyan yeni bulgular tespit edilmiştir — bunların başında SBUS alıcısının donanımsal olarak uyumsuz bir UART/GPIO eşlemesine bağlanmış olması ve etkinleştirilen watchdog'un, tasarımı gereği Çekirdek 1'deki uçuş kontrol döngüsünün kilitlenmesini tespit edemeyecek bir 'kör nokta' içermesi gelmektedir. Bölüm 11'de bu bulgular kod satırı referanslarıyla belgelenmiştir.

1.2 Güçlü Yönler

- PIO tabanlı, jitter'siz donanımsal servo PWM üretimi (Output.cpp) — CPU'yu meşgul etmeyen hassas zamanlama.
- MPU6050 için ham I2C + DMA ping-pong okuma mimarisi — CPU sensör verisini beklemiyor.
- Madgwick filtresi (9-DOF, kuaterniyon tabanlı) + sıcaklık kompanzasyonlu jiroskop drift düzeltmesi.
- Cascaded (kademeli) PID mimarisi: açığa dış döngüsü → rate iç döngüsü, endüstri standardına uygun.
- Lock-free ring buffer ile çekirdekler arası veri paylaşımı (tasarım niyeti doğru, ancak Bölüm 3.4'te detaylandırılan bir kullanım hatası içeriyor).

- MAVLink v2 parser + sender, PIO tabanlı yazılımsal UART üzerinden — donanımsal UART portları GPS ve RC için serbest bırakılıyor.
- SBUS protokolü desteği, çift katmanlı failsafe (protokol biti + 500 ms zaman aşımı).
- picoport2'de YENİ: watchdog artık etkin, stack-overflow / malloc-failed hook'ları tanımlı, callback tabanlı MAVLink-FlightManager ayrıştırması.

1.3 v0.2.0 → picoport2: Değişim Takip Tablosu

Haziran 2026 tarihli önceki incelemede işaretlenen 13 bulgunun (3 BUG + 7 WARN + 3 düşük öncelik) picoport2 dalındaki güncel durumu aşağıda özetlenmiştir.

Önceki Bulgu (Haz. 2026)	picoport2 Durumu	Kanıt (kod)
BUG-001: Watchdog hiç başlatılmıyor	ÇÖZÜLDÜ (yeni kör nokta ile)	main.cpp: watchdog_enable(2000,true) + watchdog_update()
BUG-002: updateIMU() sadece gyro entegrasyonu	ÇÖZÜLDÜ	SensorFusion.cpp: ivme tabanlı çapraz-çarpım düzeltmesi eklendi
BUG-003: Heap değeri hardcode (182000)	ÇÖZÜLDÜ	main.cpp: rp2040.getFreeHeap()
WARN-003: Mixer.h kullanılmıyor (ölü kod)	DEĞİŞMEDİ	src/core/Mixer.cpp/h hâlâ mevcut, hiçbir yerden çağrılmıyor
WARN-004: IMU.cpp kullanılmıyor (ölü kod)	DEĞİŞMEDİ	src/drivers/IMU.cpp/h hâlâ mevcut
WARN-005: PID anti-windup yetersiz (sade clamp)	İYİLEŞTİRİLDİ	PID.cpp: satürasyon yönünde koşullu entegrasyon geri alımı
WARN-006: MavlinkHandler → FlightManager sıkı bağlılık	ÇÖZÜLDÜ	main.cpp: setFlightDataProvider/setRCOverrideHandler callback'leri
WARN-010: RC_CHANNELS_OVERRIDE işlenmiyor	ÇÖZÜLDÜ	FlightManager::setRCOverride()/clearRCOverride() uçtan uca bağlı
WARN-011: Param değişikliği PID'e yansımıyor	DEĞİŞMEDİ (kapsam dışı kaldı)	ParamManager derlemede kapalı (#ifdef), ayrıca hiç çağrılmıyor
WARN-012 / 013: RC_Values, Serial.h/.cpp boş dosyalar	DEĞİŞMEDİ	types.h, Serial.h/.cpp hâlâ boş/ölü
Stack overflow koruması yok	ÇÖZÜLDÜ	vApplicationStackOverflowHook() + configCHECK_FOR_STACK_OVERFLOW=2
Sensör kalibrasyonu yok	DEĞİŞMEDİ	Sensors.cpp'de bias/kalibrasyon rutini yok
Kalıcı parametre depolama yok	DEĞİŞMEDİ (kapsam dışı kaldı)	ParamManager pasif olduğu için gündem dışı

i Metodolojik Not

Yukarıdaki tablo yalnızca önceki raporla örtüşen kelimeleri kapsar. Bölüm 11'de listelenen picoport2'ye özgü yeni bulgular (SBUS GPIO uyumsuzluğu, watchdog kör noktası, RingBuffer çoklu tüketici yarış durumu, boot sağlık raporunun sabit-doğru değerler döndürmesi, sensör isim uyumsuzluğu vb.) bu karşılaştırmanın dışındadır çünkü önceki raporda hiç ele alınmamıştır.

1.4 Kritik Eksikler ve Riskler (Güncel Durum)

En Yüksek Öncelik — Uçuşu Engelleyebilir

RX.cpp içinde SBUS alıcısı 'Serial2' (RP2350 Arduino çekirdeğinde UART1'e karşılık gelir) nesnesi üzerinden GPIO1'e bağlanıyor. Ancak RP2040/RP2350 pin fonksiyon tablosunda GPIO1'in F2 (UART) fonksiyonu yalnızca UART0 RX'tir — UART1 için kullanılabilir değildir. Kodun kendi yorum satırı bile bunu doğruluyor: '// Serial2 → GP1 (RX)' yorumunun hemen üstünde config.h '#define PIN_SBUS_RX 1 // UART0 RX' notunu taşıyor. Bu, donanımda RC alıcısının hiç veri almaması riskini taşıyan, doğrulanması gereken kritik bir bulgudur (bkz. Bölüm 9.5 ve BUG-101).

Kritik — Güvenlik Mimarisinde Kör Nokta

Watchdog artık etkin olsa da, watchdog_update() çağrısı hem düşük öncelikli TelemetryTask'ta (Core 0, her 20 ms) hem de gerçek uçuş kontrol döngüsü olan core1_entry()'de (Core 1) yapılıyor. RP2350'nin tek, paylaşılan donanım watchdog sayacı, HERHANGİ bir çekirdekten gelen bir 'besleme' ile sıfırlanır. Bu nedenle Core 1'deki PID+Mixer döngüsü tamamen kilitlense (sonsuz döngü, deadlock, mixer içinde tanımsız davranış) bile, Core 0'daki TelemetryTask watchdog'u beslemeye devam ettiği sürece sistem resetlenmez — uçak, donmuş/bayat kontrol çıkışlarıyla uçmaya devam eder. Bu, watchdog'un asıl amacını (kritik döngünün canlılığını garanti etmek) fiilen geçersiz kılan bir mimari tasarım hatasıdır.

- Sensör kalibrasyonu (gyro/accel bias) hâlâ yok — her açılışta sıfır ofset varsayılıyor.
- Boot sağlık raporu (BootLogger::printHealthReport çağrıları), gerçek sensör durumunu sorgulamadan sabit 'true' değerleri basıyor — pilotu yanlış güvenlik hissine sürükleyebilir (Bölüm 7.7, BUG-104).
- ParamManager derleme zamanında devre dışı ve MavlinkHandler mesaj yönlendiricisine hiç bağlı değil; etkinleştirilse bile PARAM_SET mesajları işlenmeyecektir.
- Fren/kaçınma (brownout) algılaması ve güvenli mod (safe mode) mimarisi hâlâ yok.
- Test altyapısı (host-side unit test, HIL, SITL) hâlâ mevcut değil.

1.5 Profesyonellik Puanı (Genel Özet)

Kriter	Puan	Not
Mimari Tasarım	8/10	Katmanlı yapı korunmuş; çekirdekler arası veri paylaşımında kavramsal tutarsızlık var
Gerçek Zamanlı Uygunluk	6/10	500 Hz döngü sağlam; watchdog kör noktası ve tekil-üretici varsayımının ihlali puanı düşürüyor
Sensör Katmanı	6/10	DMA mimarisi iyi; kalibrasyon yok, boot raporu güvenilir değil, isimlendirme GY-87 bileşenleriyle uyuşmuyor

Kriter	Puan	Not
Sensor Fusion	7/10	9-DOF Madgwick tam; 6-DOF yolu artık ivme düzeltmesi içeriyor ama iki yol farklı algoritma kullanıyor
Kontrol Algoritmaları	7/10	Cascaded PID + geliştirilmiş anti-windup; feed-forward hâlâ yok
Güvenlik / Failsafe	5/10	RC failsafe sağlam; watchdog kör noktası ve SBUS pin riski ciddi düşüşe neden oluyor
Haberleşme	7/10	MAVLink akışı ve RC override artık tam; parametre sistemi devre dışı
Kod Kalitesi / Bakım	6/10	Okunabilir ve modüler; ölü kod birikimi (4 modül) ve isimlendirme tutarsızlıkları var
Test / Doğrulama	2/10	Test altyapısı hâlâ yok
Ticari Potansiyel	6/10	v0.2.0'a göre ilerleme somut; SBUS ve watchdog bulguları giderilmeden uçuşa hazır değil

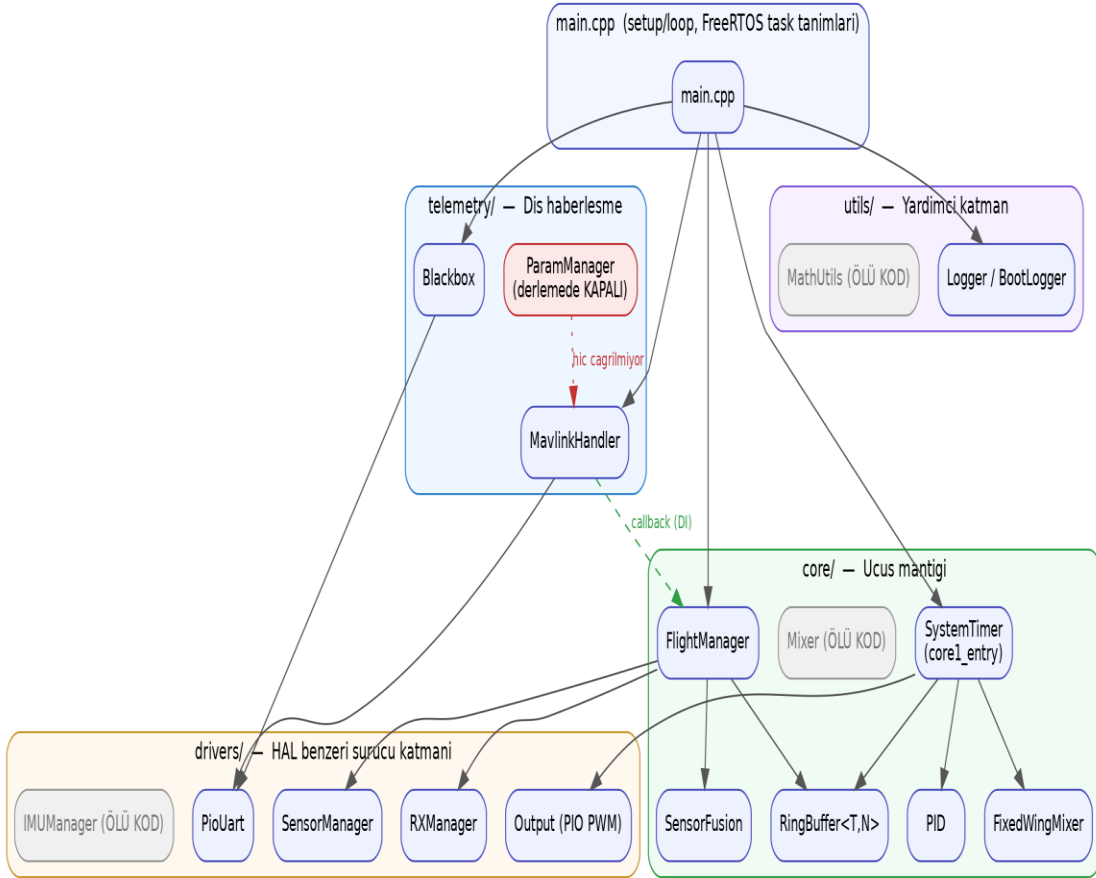
1.6 Ticari Potansiyel

Mevcut haliyle proje; akademik araştırma, gömülü sistem eğitimi ve firmware geliştirme referansı olarak yüksek değer taşımaya devam etmektedir. picoport2 dalı, watchdog ve stack-overflow koruması gibi üretim kalitesine yaklaşan iyileştirmeler eklemiştir. Ancak bu raporda tespit edilen SBUS GPIO uyumsuzluğu ve watchdog kör noktası giderilmeden, gerçek bir uçuş denemesi için 'hazır' olarak nitelendirilmesi önerilmez. Bu iki madde giderildikten sonra, proje küçük ölçekli hobi/prototip UAV kullanımına uygun bir olgunluk seviyesine ulaşacaktır.

BÖLÜM 2 — Yazılım Mimarisi

2.1 Katmanlı Mimari Analizi

AeroPico FC, beş belirgin katman etrafında organize edilmiştir: donanım katmanı (PIO/DMA/I2C, doğrudan Pico SDK çağrıları), sürücü katmanı (src/drivers/), çekirdek katmanı (src/core/), telemetri katmanı (src/telemetry/) ve yardımcı katman (src/utls/). Bu ayrım picoport2 dalında da korunmuştur ve okunabilirlik açısından projenin en güçlü yönlerinden biridir. Aşağıdaki diyagram, kod tabanının gerçek modül bağımlılıklarını göstermektedir — 'ÖLÜ KOD' olarak işaretlenen kutular, hiçbir yerden çağrılmayan ama depoda hâlâ duran dosyalardır.



Şekil 2.1 — Modül bağımlılık diyagramı (gerçek include/çağrı ilişkilerinden çıkarılmıştır)

2.2 Dosya Organizasyonu ve Modüler Yapı

Modül	Dosya	Satır	Sorumluluk	Durum
FlightManager	core/FlightManager.cpp/h	~106	Sensör+RX+fusion birleştirme, arm/disarm, RC override	Aktif
SensorFusion	core/SensorFusion.cpp/h	~179	Madgwick 9-DOF + 6-DOF, sıcaklık kompanzasyonu	Aktif

Modül	Dosya	Satır	Sorumluluk	Durum
SensorManager	drivers/Sensors.cpp/h	~227+82	MPU6050 DMA, HMC5883L/BMP085 (Wire)	Aktif
PID	core/PID.cpp/h	~42+18	Koşullu-entegrasyon anti-windup'lı PID	Aktif
FixedWingMixer	core/FixedWingMixer.cpp/h	~57+35	Servo karıştırma, gain/trim, PIO çıkışı	Aktif
SystemTimer	core/SystemTimer.cpp/h	~97+18	Core 1 bare-metal 500Hz döngüsü, 5xPID orkestrasyon	Aktif
Output	drivers/Output.cpp/h	~45+12	PIO tabanlı 4 kanal PWM üretimi	Aktif
RXManager	drivers/RX.cpp/h	~74+23	SBUS parse, çift katmanlı failsafe	Aktif — pin riski (\$9.5)
RingBuffer<T,N>	core/RingBuffer.h	~51	Lock-free SPSC şablon sınıfı	Aktif — yanlış kullanılıyor (\$3.4)
MavlinkHandler	telemetry/MavlinkHandler.cpp/h	~214+76	MAVLink v2 parse/gönderim, callback arayüzü	Aktif
Blackbox	telemetry/Blackbox.cpp/h	~30+24	CSV uçuş verisi loglama (PIO UART üzerinden)	Aktif
PioUart	drivers/PioUart.cpp/h	~50+26	Yazılımsal UART (PIO SM ile)	Aktif
BootLogger / Logger	utils/*.cpp/h	~66+12/16+12	Açılış raporu ve genel loglama	Aktif — güvenilirlik sorunu (\$7.7)
ParamManager	telemetry/ParamManager.cpp/h	~85+63	MAVLink PARAM protokolü	Derlemede KAPALI, yönlendirilmiyor
Mixer	core/Mixer.cpp/h	~37+15	Servo.h tabanlı alternatif mixer	ÖLÜ KOD
IMUManager	drivers/IMU.cpp/h	~26+20	Legacy complementary filter	ÖLÜ KOD
MathUtils	utils/MathUtils.cpp/h	~13+14	Açı normalize / clamp yardımcıları	ÖLÜ KOD
SerialCom	telemetry/SerialCom.cpp/h	2+6	Planlanan seri haberleşme	Boş taslak
Serial	Serial.cpp/h	0+0	—	Boş dosya

2.3 Modüller Arası Bağımlılıklar

main.cpp → FlightManager → SensorManager + RXManager + SensorFusion zinciri Core 0'da; main.cpp → SystemTimer::init() → multicore_launch_core1() → PID + FixedWingMixer zinciri Core 1'de yürütülüyor. picoport2'nin en dikkat çekici mimari iyileştirmesi, önceki raporda 'sıkı bağıllık' olarak işaretlenen MavlinkHandler → FlightManager ilişkisinin artık fonksiyon işaretçisi tabanlı bir bağımlılık enjeksiyonuna (FlightDataProvider, RCOVERRIDEHandler, RCClearHandler tipleri) dönüştürülmüş olmasıdır; MavlinkHandler.h veya .cpp içinde FlightManager'a artık doğrudan hiçbir referans yoktur.

✓İyileştirme Doğrulandı

main.cpp içinde mavlink.setFlightDataProvider(getFlightDataCallback) ve ilgili RC override/clear callback'leri kaydediliyor; MavlinkHandler.cpp'de flightManager sembolüne hiçbir yerde erişilmiyor. Bu, önceki raporun WARN-006 önerisinin harfiyen uygulandığı, nadir görülen tam bir refactor örneğidir.

2.4 SOLID Prensipleri Analizi

- Tek Sorumluluk (SRP): Her sınıf tek bir işe odaklanıyor; FlightManager biraz şişkin olsa da (sensör+RX+fusion+arm mantığı) hâlâ kabul edilebilir sınırlar içinde.
- Açık/Kapalı (OCP): MixerSettings struct'ı üzerinden genişletilebilirlik var; SensorFusion'ın update()/updateIMU() ayrımı 9-DOF/6-DOF modüler kullanım sağlıyor.
- Liskov Yerine Geçme (LSP): Kalıtım neredeyse hiç kullanılmıyor; ihlal riski yok.
- Arayüz Ayrımı (ISP): Header dosyaları küçük ve odaklı.
- Bağımlılığın Tersine Çevrilmesi (DIP): MavlinkHandler artık soyutlama (callback) üzerinden konuşuyor — picoport2'de en çok gelişen prensip bu oldu. FlightManager ise hâlâ SensorManager/RXManager/SensorFusion somut sınıflarını doğrudan içeriyor (üye değişken olarak); bu, tek donanım hedefi için kabul edilebilir ama çoklu sensör/HAL desteği hedefleniyorsa arayüz soyutlaması gerekecektir.

2.5 Tasarım Desenleri

- Ping-Pong Buffer (double buffering): SensorManager::_buf[2] — DMA yazarken okuma tarafını bloklamıyor.
- Lock-Free Queue (SPSC niyetiyle): RingBuffer<T,SIZE> — bkz. §3.4 için kullanım hatası uyarısı.
- Observer/Callback: MavlinkHandler'ın FlightDataProvider/RCOverrideHandler/RCClearHandler fonksiyon işaretçileri.
- Strategy (kısmi): SensorFusion::update() (9-DOF) / updateIMU() (6-DOF) ayrımı, USE_GY87 makrosuyla derleme zamanında seçiliyor.
- Cascaded Controller: Angle PID → Rate PID iki katmanlı kontrol yapısı.

2.6 Kod Karmaşıklığı

Toplam 2.415 satırlık kod tabanı; ortalama dosya başına ~51 satır ile son derece düşük karmaşıklıkta kalmıştır (PX4/ArduPilot'ta modül başına yüzlerce-binlerce satır olağandır). En uzun tek dosya MavlinkHandler.cpp (214 satır) olup, döngüsel karmaşıklığı (cyclomatic complexity) düşük; switch-case tabanlı mesaj yönlendirmesi dışında derin dallanma yoktur. core1_entry() (SystemTimer.cpp) fonksiyonel olarak yoğun ama doğrusal akışlı, kolay izlenebilir bir döngüdür.

2.7 Ölçeklenebilirlik Değerlendirmesi

Mevcut mimari 4 servo + 1 motor çıkışı ve tek sensör paketi için yeterlidir. Çoklu uçuş modu (MANUAL/FBWA/CRUISE), irtifa tutma veya GPS navigasyonu eklendiğinde FlightManager'ın sorumlulukları hızla aşılacaktır. Önceki raporun v0.4+ için önerdiği FlightManager'ın FlightMode/AltitudeController/NavigationController alt sınıflarına bölünmesi tavsiyesi picoport2'de henüz uygulanmamıştır ve güncelliğini korumaktadır.

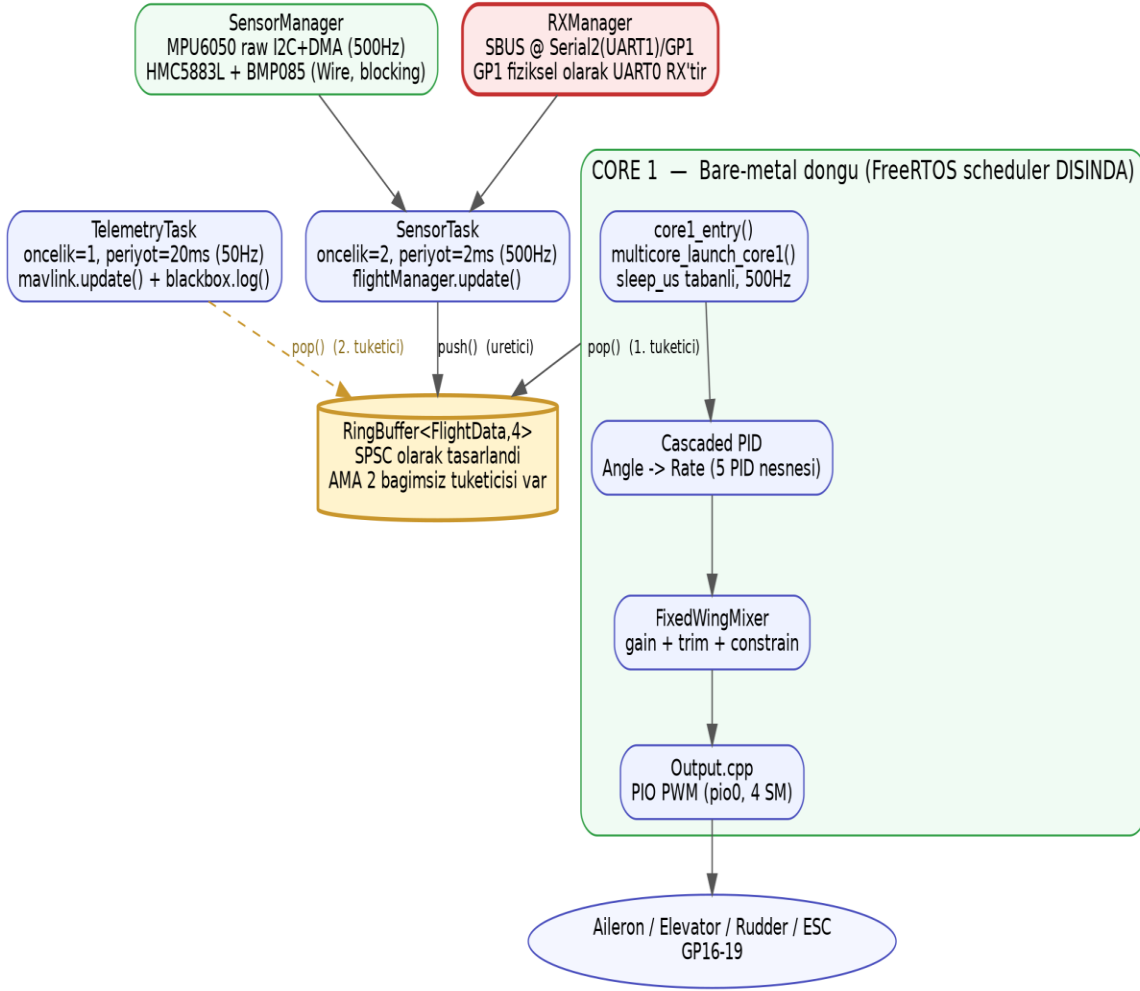
BÖLÜM 3 — Gerçek Zamanlı Sistem Analizi

3.1 FreeRTOS Görev Mimarisi

main.cpp, setup() içinde iki FreeRTOS görevi oluşturuyor (xTaskCreateAffinitySet) ve ardından vTaskStartScheduler() çağırıyor. Bu iki görev de affinity maskesi (1 << 0) ile yalnızca Core 0'a sabitlenmiştir. Buna paralel olarak SystemTimer::init(), FreeRTOS scheduler başlamadan önce multicore_launch_core1() ile Core 1'de tamamen bare-metal (RTOS dışı) bir döngü başlatır. Sonuç olarak gerçek mimari, README.md'deki 'Core 1: FreeRTOS Task, higher priority' ifadesiyle örtüşmemektedir — Core 1'de FreeRTOS scheduler'ı hiç çalışmaz.

Görev / Döngü	Çekirdek	Öncelik	Periyot	Stack	Yürütme Modeli
SensorTask	Core 0	2 (yüksek)	2 ms (500 Hz)	2048 word	FreeRTOS task, vTaskDelayUntil
TelemetryTask	Core 0	1 (düşük)	20 ms (50 Hz)	2048 word	FreeRTOS task, vTaskDelayUntil
core1_entry (PID+Mixer)	Core 1	—	2 ms (500 Hz)	N/A (ayrı yığın alanı)	Bare-metal, sleep_us tabanlı

3.2 Core Affinity ve Görev Zamanlaması



Şekil 3.1 — Gerçek çekirdek/görev mimarisi (README'deki iddiadan farklı olarak Core 1 FreeRTOS dışıdır)

SensorTask ve TelemetryTask'ın her ikisinin de Core 0'da olması, determinizm açısından savunulabilir bir tercihtir: vTaskDelayUntil() doğru kullanılmış, öncelik farkı ($2 > 1$) 500 Hz sensör döngüsünün 50 Hz telemetri döngüsünü kesintiye uğratabilmesini (preemption) sağlıyor ve saat kayması (drift) oluşmuyor. Ancak bu tasarım, aşağıda ayrıntılandırılan iki önemli yan etkiyi de beraberinde getiriyor: (1) TelemetryTask, RingBuffer'ın ikinci bağımsız tüketicisi hâline geliyor (§3.4) ve (2) watchdog'un Core 1'in canlılığını garanti etmesini engelliyor (§3.7).

3.3 Mutex ve Senkronizasyon

SensorManager içinde Pico SDK'nın donanımsal mutex_t türü, ping-pong sensör tamponunu (_buf[2]) korumak için doğru şekilde kullanılıyor (mutex_init, mutex_enter_blocking, mutex_exit). Bunun dışında kod tabanında (grep taraması ile doğrulanmıştır) hiçbir xSemaphore*, portENTER_CRITICAL veya başka bir senkronizasyon ilkelisi bulunmamaktadır — README.md'nin iddia ettiği 'Mutex — Cross-core data protection' ifadesi, gerçekte RingBuffer'ın lock-free tasarımına karşılık gelmektedir; ayrı bir mutex mekanizması yoktur.

⚠️Taşınan Bulgu

`mutex_enter_blocking()`, FreeRTOS bağlamında çağrıldığında (`SensorManager::getLatest()` `SensorTask` içinden çağrılıyor) donanım seviyesinde meşgul-bekleme (`busy-wait`) yapar; bu, FreeRTOS'un kendi `xSemaphoreTake()` ilkelinden farklı olarak scheduler'a görev değiştirme fırsatı vermez. Kritik bölge çok kısa olduğu için (basit struct kopyası) pratik etkisi düşüktür, ancak ölçeklenirse (örn. daha büyük sensör paketleri) gecikme kaynağı olabilir.

3.4 Lock-Free Ring Buffer (SPSC) Analizi — Kritik Bulgu

`RingBuffer<FlightData,4>`, yorum satırında açıkça 'Single Producer Single Consumer (SPSC)' olarak tanımlanmış ve `__dmb()` bellek bariyeriyle güçlendirilmiştir. Tasarımın kendisi (`volatile head/tail`, DMB bariyeri) SPSC senaryosu için doğrudur. Ancak kod tabanı taraması, bu varsayımın `picoport2`'de ihlal edildiğini göstermektedir: `_ringBuf` yalnızca `SensorTask` tarafından `push()` ediliyor (tek üretici, doğru), fakat `pop()` işlemi `HEM core1_entry()` (Core 1) `HEM DE taskTelemetry()` içindeki `flightManager.getLatestData()` çağrısı (Core 0) üzerinden İKİ bağımsız yerden tetikleniyor.

🐛 BUG — Çoklu Tüketici Yarış Durumu (Race Condition)

`core/RingBuffer.h` SPSC olarak tasarlanmıştır ve `_tail` indeksinin tek bir tüketici tarafından güncelleneceğini varsayar. Ancak `main.cpp`'deki `taskTelemetry()` (Core 0, mavlink/blackbox verisi için) ve `core/SystemTimer.cpp`'deki `core1_entry()` (Core 1, kontrol döngüsü için) her ikisi de `FlightManager::getLatestData()` üzerinden aynı ring buffer'ı bağımsız olarak `pop()` ediyor. İki tüketici aynı anda (biri Core 0 kesmesiyle, diğeri Core 1'de) çalışırsa, `_tail` üzerinde parçalanmış okuma-değiştir-yaz (`read-modify-write`) sırası oluşabilir: bir tüketici diğerrinin henüz işlemediği bir veri paketini 'yutabilir' ya da her iki tüketici de aynı elemanı okuyup `_tail`'i iki kez ilerletebilir, bu da `FlightManager::_latest` önbelleğinin çekirdekler arasında öngörülemez şekilde eski/güncel karışık veri döndürmesine yol açabilir. Pratik sonucu, kontrol döngüsünün bazı örneklerde beklenenden bir tık eski/yeni veri kullanması olsa da, tasarımın SPSC belgeleriyle gerçek kullanım şekli arasındaki uyumsuzluk düzeltilmelidir.

Önerilen düzeltme: `RingBuffer`'ı gerçek bir SPSC olarak kullanmak için tek tüketici belirlenmeli (örneğin yalnızca Core 1) ve `TelemetryTask`, `FlightManager` üzerinde ayrı, salt-okunur bir 'peek/cache' arayüzü kullanmalı (`RingBuffer::peek()` zaten mevcut ve tam da bu amaç için tasarlanmış görünüyor, ancak şu an kullanılmıyor); ya da yapı MPSC (multi-producer/multi-consumer) için uygun bir tasarıma (örn. gerçek mutex korumalı çift tampon) yükseltilmelidir.

3.5 PIO State Machine Analizi

Proje `pio0`'ı 4 PWM servo state machine'i (aileron, elevator, rudder, throttle) için, `pio1`'i ise 2 UART state machine'i (TX, RX — ESP32-CAM bağlantısı) için kullanıyor. `RP2350`'de her PIO bloğu 4 SM içerir, toplamda 8 SM mevcuttur; 6 SM kullanımı (%75) limitlere yakın ama makuldür ve GPS UART'ının donanımsal UART1 üzerinden ayrı tutulması PIO kapasitesini korumaktadır.

PIO Bloğu	SM	Program	Pin	Amaç
pio0	0	pwm_servo	GP16 (Aileron)	Servo PWM
pio0	1	pwm_servo	GP17 (Elevator)	Servo PWM

PIO Bloğu	SM	Program	Pin	Amaç
pio0	2	pwm_servo	GP18 (Rudder)	Servo PWM
pio0	3	pwm_servo	GP19 (Throttle/ESC)	Servo PWM
pio1	0	pio_uart_tx	GP12 (ESP TX)	Yazılımsal UART
pio1	1	pio_uart_rx	GP13 (ESP RX)	Yazılımsal UART

3.6 Worst Case Execution Time (WCET) Tahmini

500 Hz döngü bütçesi 2000 μ s'dir. Aşağıdaki tahminler, RP2350 @133MHz varsayılan saat hızı ve donanım FPU dikkate alınarak, kod yapısından (döngü sayısı, çağrı derinliği, blocking/non-blocking karışımı) türetilmiştir; gerçek profil için lojik analizör/osiloskop ölçümü önerilir.

İşlem	Tahmini Süre	Not
DMA tetikleme (I2C RX)	~5 μ s	CPU'yu bloklamıyor
I2C register-adres yazımı (DMA öncesi)	~15-25 μ s	i2c_write_blocking — hâlâ blocking (§4.6)
Madgwick update() / updateIMU()	~120-180 μ s	Float ops, FPU hızlandırma
Cascaded PID (5 nesne)	~15-25 μ s	Basit float aritmetik
FixedWingMixer::compute()	~8-12 μ s	map() + constrain()
MAVLink byte-by-byte parse	~20-40 μ s	Yalnızca gelen veri varsa
Blackbox snprintf	~40-60 μ s	String biçimlendirme maliyetli (TelemetryTask içinde, 20ms bütçeli)
Wire.h üzerinden mag/baro okuma	~2-5 ms (en kötü)	Blocking I2C — 500Hz bütçesini aşabilir, bkz. §4.6

⚠WCET Sonucu

SensorTask döngüsünün çekirdek maliyeti (DMA + Madgwick + PID + Mixer) 2000 μ s bütçenin yaklaşık %10-15'i kadardır ve rahat bir marj bırakır. Ancak mag/baro okumaları (Wire.h, blocking) en kötü senaryoda birkaç milisaniyeye çıkabilir; bu, SensorTask'ın 2 ms periyoduna göre BÜYÜK bir aşımır ve dönemsel olarak döngü gecikmesine (jitter) neden olabilir. Bu risk zaten önceki raporda WARN-002 olarak işaretlenmişti ve picoport2'de giderilmemiştir.

3.7 Watchdog ve Priority Inversion — Yeniden Değerlendirme

Önceki inceleme, watchdog'un hiç etkinleştirilmediğini tespit etmişti (BUG-001). picoport2'de watchdog_enable(WATCHDOG_TIMEOUT_MS, true) artık main.cpp'nin sonunda çağrılıyor ve watchdog_update() üç ayrı yerde tetikleniyor: taskTelemetry() (her 20 ms), core1_entry()'nin armed dalının sonunda (her 2 ms, yalnızca sistem arm edilmişken) ve main.cpp'deki hata kancalarında (stack overflow / malloc failed). Bu, ilk bakışta tam bir çözüm gibi görünse de, aşağıdaki mantık hatası kalıcıdır:

```
// core/SystemTimer.cpp — core1_entry() if (!flightManager.isArmed()) {
writeMotors(PWM_MIN, PWM_NEUTRAL, PWM_NEUTRAL, PWM_NEUTRAL);    continue;           // <--
watchdog_update()'e hiç ulaşmıyor } ... watchdog_update();       // yalnızca armed
durumdayken çalışır
```


Sistem disarm iken Core 1'in watchdog'u hiç beslememesi başlı başına önemli bir bulgudur, fakat asıl mimari sorun daha derindir: RP2350'de watchdog tek, global bir donanım sayacıdır ve hangi çekirdekten geldiğine bakılmaksızın herhangi bir watchdog_update() çağrısı sayaçları sıfırlar. taskTelemetry() (Core 0, düşük öncelik, yalnızca telemetri/blackbox işi yapar) her 20 ms'de bir bu global sayacı sıfırlamaya devam ettiği sürece, Core 1'in tamamen kilitlenmiş olması (örneğin core1_entry() içinde sonsuz döngüye giren bir hata, ya da PID/Mixer içinde NaN/Inf yayılımı sonucu oluşan tanımsız davranış) sistemi resetlemeyecektir. Watchdog, tasarımı gereği 'uçuş kritik döngü canlı mı?' sorusunu değil, 'herhangi bir görev hâlâ CPU zamanı buluyor mu?' sorusunu yanıtlamaktadır — bu ikisi aynı şey değildir.

Önerilen düzeltme: Ya (a) ayrı bir 'Core 1 canlılık' watchdog'u yazılımsal olarak uygulanmalı (Core 1, paylaşılan bir zaman damgasını her turda güncellemeli; Core 0 bu damgayı izleyip belirli bir eşik aşıldığında sistemi kasıtlı olarak resetlemeli), ya da (b) watchdog_update() yalnızca Core 1'den çağrılmalı ve Core 0'ın sağlıklı olduğu ayrı bir mekanizma (örn. Core 1'in Core 0'ı izlemesi) ile doğrulanmalıdır.

BÖLÜM 4 — Sensör Katmanı Analizi

4.1 MPU6050 Sürücüsü — Ham I2C + DMA

Sensors.cpp, Arduino Wire kütüphanesi yerine doğrudan Pico SDK'nın `i2c_init()`, `i2c_write_blocking()` ve DMA API'sini kullanıyor. WHO_AM_I doğrulaması (register 0x75, beklenen 0x68) `init()` içinde yapılıyor ve başarısız olursa `_imuAvailable = false` ile sensör devre dışı bırakılıyor — bu, savunmacı programlamanın iyi bir örneğidir ve `picoport2`'de korunmuştur.

Parametre	Değer	Değerlendirme
İvmeölçer aralığı	$\pm 8g$ (ACCEL_CFG=0x10), ölçek 1/4096 g/LSB	Uçuş için uygun
Jiroskop aralığı	$\pm 500^\circ/s$ (GYRO_CFG=0x08), ölçek 1/65.5 $^\circ/s/LSB$	Sabit kanat için ideal
DLPF bant genişliği	21 Hz (CONFIG=0x04)	Servo titreşimini filtreler, ~13ms gecikme — kabul edilebilir
Sıcaklık formülü	$T = \text{ham}/340 + 36.53^\circ C$	MPU6050 datasheet ile uyumlu
IIR alpha (ivme)	0.15 $\rightarrow$ ~13 Hz kesim (500Hz örnekleme)	Titreşim gürültüsünü makul ölçüde bastırır
Jiroskop filtresi	Yok (kasıtlı)	Doğru karar — Madgwick zaten entegrasyon yapıyor

4.2 DMA Pipeline'ının Tamlığı

`_mpu_start_dma_read()` akışı: register adresini yaz $\rightarrow$ I2C'ye 14 baytlık okuma komutu gönder $\rightarrow$ DMA, I2C RX FIFO'dan RAM'e aktarır. Bu, CPU'yu veri bekleme süresinden kurtarır. Ancak register adresini yazan `i2c_write_blocking()` çağırısı hâlâ bloklayıcıdır — tam bir non-blocking pipeline için I2C TX tarafı da DMA'ya alınmalıdır. Bu, önceki raporda WARN-001 olarak işaretlenmişti; `picoport2`'de değişmemiştir. Etkisi küçüktür ($<100 \mu s$) ama `SensorTask`'ın 2 ms bütçesi içinde birikimli bir gecikme kaynağıdır.

4.3 GY-87 Modülü — İsimlendirme Uyuşmazlığı (Yeni Bulgu)

`config.h` içinde `USE_GY87` makrosu etkinken, `Sensors.h` Adafruit_HMC5883_Unified ve Adafruit_BMP085_Unified sürücülerini dahil ediyor ve `Sensors.cpp` bu iki sensörü `Wire.h` üzerinden başlatıyor (`mag.begin()`, `bmp.begin()`). Bu, gerçek GY-87 modülünün standart bileşenleriyle (HMC5883L manyetometre + BMP085 barometre) birebir örtüşüyor.

⚠️BUG-102 — Boot Raporu Yanlış Sensör Adları Basıyor

`main.cpp:setup()` içinde `BootLogger::ok("BMP280")` ve `BootLogger::ok("QMC5883L")` satırları, gerçekte derlenen ve başlatılan sensörlerin (BMP085, HMC5883L) isimleriyle uyuşmuyor. BMP280 ve QMC5883L, HMC5883L/BMP085'in modern muadilleridir ancak register haritaları ve bazı ölçüm karakteristikleri farklıdır — bu isimler muhtemelen gelecekteki bir donanım revizyonunu yansıtan, güncellenmemiş bir yer tutucudur. Kod incelemesi ile boot log arasındaki bu tutarsızlık, hata ayıklama sırasında yanlış varsayımlara yol açabilir ve düzeltilmelidir (bkz. §7.7 için ilişkili, daha ciddi bir bulgu).

4.4 Blocking I2C — Mag/Baro Okumaları

mag.getEvent() ve bmp.getEvent() çağrıları Adafruit BusIO üzerinden Wire.h kullanır ve blocking I2C yapar. Bu çağrılar SensorManager::update() içinde, DMA ile okunan MPU6050 verisinden hemen sonra senkronize şekilde çalıştırılıyor; en kötü senaryoda SensorTask'ı birkaç milisaniye geciktirebilir (bkz. §3.6 WCET tablosu). Bu bulgu önceki raporda WARN-002 olarak işaretlenmişti ve picoport2'de değişmemiştir.

4.5 Sensör Doğrulama Özeti

Sensör	Doğrulama	Hata İşleme	Kalibrasyon	Boot Log Tutarlılığı
MPU6050	WHO_AM_I = 0x68 ✓	fail() + _imuAvailable=false ✓	Yok ✗	Doğru isim, ama sabit true (§7.7)
HMC5883L	mag.begin() ✓	hasMag=false ✓	Yok ✗	'QMC5883L' olarak yanlış raporlanıyor
BMP085	bmp.begin() ✓	hasBaro=false ✓	Yok ✗	'BMP280' olarak yanlış raporlanıyor

4.6 Kalibrasyon Eksikliği

⚠Değişmedi

MPU6050 için ivmeölçer/jiroskop bias kalibrasyonu hâlâ yok. İlk açılışta sensör ofsetleri sıfır kabul ediliyor; gerçek uçuşta drift ve sabit açısal hata oluşacaktır. En azından N-örnek ortalaması ile statik bias hesaplama (boot sırasında, uçak sabitken) eklenmesi önerilir — bu, PX4/ArduPilot'ta zorunlu bir ön uçuş adımıdır.

BÖLÜM 5 — Sensor Fusion Analizi

5.1 Madgwick Filtresi — 9-DOF update()

SensorFusion::update(), Sebastian Madgwick'in orijinal algoritmasının tam gradyan-inişi (gradient descent) implementasyonunu içeriyor: ivmeölçer + manyetometre ile hata fonksiyonu (f1..f6), Jakobyen (J_11..J_64), normalize edilmiş düzeltme adımı (s0..s3) ve kuaterniyon entegrasyonu. Beta parametresi init() içinde 0.08 olarak ayarlanmış (FlightManager::init() → fusion.init(0.08f)); bu, filtre kararlılığı ile drift düzeltmesi arasında makul bir denge sağlar.

5.2 6-DOF updateIMU() — Önceki Bulgu Çözüldü

Önceki inceleme (BUG-002), updateIMU()'nun yalnızca jiroskop entegrasyonu yaptığını ve ivmeölçer düzeltmesinin tamamen eksik olduğunu tespit etmişti. picoport2'de bu fonksiyon güncellenmiş ve artık bir ivmeölçer tabanlı düzeltme adımı içeriyor:

```
float vx = 2.0f * (q1*q3 - q0*q2); float vy = 2.0f * (q0*q1 + q2*q3); float vz = q0*q0 - q1*q1 - q2*q2 + q3*q3; float ex = (ay*vz - az*vy); float ey = (az*vx - ax*vz); float ez = (ax*vy - ay*vx); gx += beta * ex; gy += beta * ey; gz += beta * ez;
```

✓İyileştirme Doğrulandı

Bu blok, tahmini yerçekimi vektörü ile ölçülen ivme arasındaki çapraz çarpımı (cross product) hataya dönüştürüp jiroskop okumasına ekleyen klasik bir Mahony-tipi tamamlayıcı (complementary) düzeltmedir. Artık ivmeölçer verisi düz uçuş dışındaki durumlarda da açi tahminine katkı sağlıyor; önceki raporun BUG-002 bulgusu fiilen giderilmiştir.

⚠Yeni Gözlem — Algoritma Tutarsızlığı

update() (9-DOF) tam gradyan-inişi Madgwick algoritmasını kullanırken, updateIMU() (6-DOF) çapraz çarpım tabanlı Mahony-stili bir düzeltme kullanıyor. İki fonksiyon matematiksel olarak farklı yakınsama davranışlarına sahiptir; aynı beta parametresi (fusion.init() ile tek bir değer olarak ayarlanıyor) her iki algoritma için de aynı 'anlamı' taşımaz — Mahony'de beta genellikle bir kazanç (Kp) olarak yorumlanırken Madgwick'te adım büyüklüğü katsayısıdır. USE_GY87 tanımlı değilken (yalnızca MPU6050) sisteme geçiş yapılırsa, aynı beta=0.08 değeri iki farklı filtrenin farklı davranmasına neden olabilir. Tutarlılık için her iki yol da aynı algoritma ailesinden (ör. her ikisi de gradyan-inişi) türetilmesi veya beta'nın yola özgü ayrı sabitler olarak tanımlanması önerilir.

5.3 Sıcaklık Kompanzasyonu

_gyroTempCoeff = 0.004 katsayısı ile jiroskop okumaları, referans 25°C'ye göre sıcaklık farkı kadar düzeltiliyor (tempOffset = (tempC-25)*0.004, gx/gy/gz'den çıkarılıyor). Bu, MPU6050 datasheet'iyle uyumlu bir yaklaşımdır ve v0.2 seviyesi için ileri bir özellik olmaya devam ediyor; ancak katsayı sabit ve bireysel sensöre göre kalibre edilmiyor.

5.4 Kuaterniyon → Euler Dönüşümü ve Gimbal Lock — Yeni Bulgu

```
roll = atan2(2*(q0*q1+q2*q3), 1-2*(q1*q1+q2*q2)) * 180/PI; pitch = asin(2*(q0*q2 - q3*q1)) * 180/PI; // <-- domain kontrolü yok yaw = atan2(2*(q0*q3+q1*q2), 1-2*(q2*q2+q3*q3)) * 180/PI;
```

△BUG-103 — asin() Argümanı Sınırlanmıyor

computeAngles() içinde asin() fonksiyonuna verilen argüman [-1, 1] aralığına sınırlanmıyor (clamp edilmiyor). Kuaterniyon normalizasyonu teorik olarak bu argümanı sınırlar içinde tutar, ancak float yuvarlama hataları birikimi (özellikle pitch $\pm 90^\circ$ 'ye yaklaşırken, yani gimbal lock bölgesinde) argümanın 1.0000001 gibi sınırı çok az aşmasına neden olabilir; bu durumda asin() NaN döndürür ve NaN, pitch → PID hata hesabı → mixer → servo PWM zincirine yayılarak kontrol çıkışlarını bozabilir. Düzeltme tek satırlık ve düşük risklidir: asin(constrain(2*(q0*q2-q3*q1), -1.0f, 1.0f)).

5.5 invSqrt() Fonksiyonu

SensorFusion.cpp'de invSqrt() tanımlı ancak implementasyonu basitçe $1.0f/\sqrt{x}$ 'tir; Quake III'ün ünlü bit-manipülasyon hilesi kullanılmamış. RP2350'nin donanımsal FPU'su ile native sqrt() zaten hızlı olduğundan bu, pratik bir performans sorunu değildir — ancak fonksiyonun kodda hâlâ bulunması ve hiçbir yerden çağrılmaması (grep ile doğrulanmıştır) onu bir başka küçük ölü kod parçası yapmaktadır.

5.6 Gürültü ve Drift Değerlendirmesi

Kaynak	Etki	Mevcut Çözüm	Eksik
Gyro drift	Yaw drift	Madgwick/Mahony gradyan düzeltmesi	Manyetometre kalibrasyonu
İvme titreşimi	Pitch/roll gürültüsü	IIR filtre (alpha=0.15)	Notch filtre
Manyetik bozulma	Yaw hatası	Madgwick mag fusion	Hard/soft-iron kompanzasyonu
Sıcaklık drifti	Gyro bias	Sıcaklık kompanzasyon katsayısı	Bireysel sensör kalibrasyonu
Barometre gürültüsü	Yükseklik tahmini	Yok	LPF veya Kalman filtresi

BÖLÜM 6 — Kontrol Algoritmaları Analizi

6.1 PID Kontrolcüsü ve Anti-Windup — İyileştirme Doğrulandı

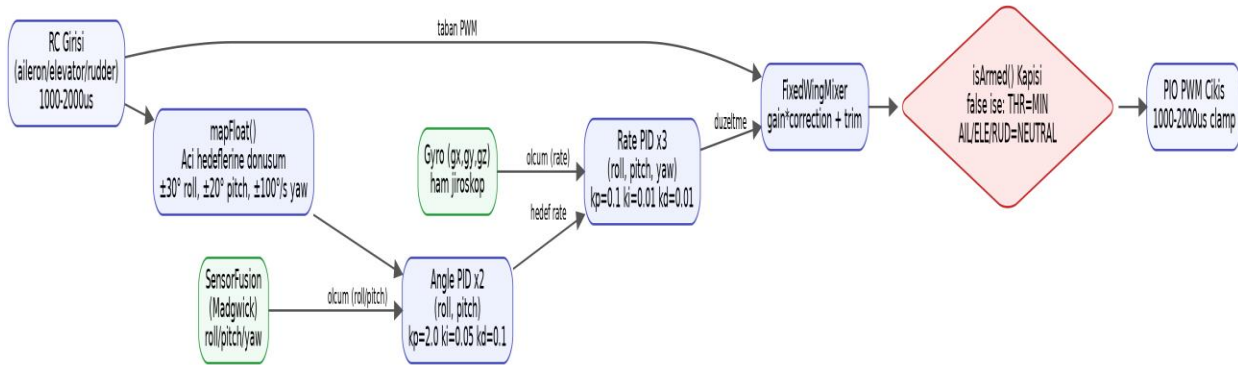
PID.cpp, __not_in_flash_func() ile RAM'de çalışacak şekilde işaretlenmiş, standart P/I/D hesaplamasının ötesinde artık koşullu entegrasyon (conditional integration) tabanlı bir anti-windup uyguluyor:

```
float candidateIntegral = integral + error * dt; candidateIntegral = constrain(candidateIntegral, -100.0f, 100.0f); float I = ki * candidateIntegral; float output = P + I + D; if (output > outputMax) { if (error > 0.0f) { candidateIntegral -= error*dt; ... } // satürasyonu büyüten yönde ise geri al output = outputMax; } else if (output < outputMin) { /* simetrik */ } if (output > outputMin && output < outputMax) integral = candidateIntegral;
```

✓İyileştirme Doğrulandı

Önceki inceleme (WARN-005), entegral terimin yalnızca ± 100 ile clamp edildiğini ve çıkış satüre olduğunda entegralin büyümeye devam ettiğini (klasik windup) tespit etmişti. picoport2'deki mevcut implementasyon, çıkış sınıra çarptığında VE hata satürasyonu daha da büyütecek yönde ise entegral artışını geri alıyor (koşullu entegrasyon / back-calculation'a yakın bir yaklaşım). Bu, PX4'ün clamp+integral_clamp yaklaşımına kavramsal olarak yakın, sağlam bir çözümdür.

6.2 Cascaded PID Mimarisi



Şekil 6.1 — Cascaded (kademeli) kontrol döngüsü veri akışı

SystemTimer.cpp içinde 5 PID nesnesi tanımlı: rollAnglePID / pitchAnglePID (dış döngü, açı→rate hedefi) ve rollRatePID / pitchRatePID / yawRatePID (iç döngü, rate→servo düzeltmesi). Yaw eksenini yalnızca rate kontrolü kullanıyor — sabit kanatlı uçaklar için doğru bir tercih, çünkü yaw genelde koordineli dönüş için ikincil bir eksenidir. Bu cascaded yapı, PX4 ve ArduPilot'un FBWA (Fly-By-Wire-A) modlarıyla aynı temel mantığı izler.

6.3 RC Giriş İşleme

RC kanalları 1000-2000µs aralığından fiziksel birimlere mapFloat() ile dönüştürülüyor: aileron → $\pm 30^\circ$ (MAX_ROLL_ANGLE), elevator → $\pm 20^\circ$ (MAX_PITCH_ANGLE), rudder → $\pm 100^\circ/\text{s}$ (MAX_YAW_RATE). Bu üst sınırlar config.h'da merkezi olarak tanımlı ve uçuş testine göre ayarlanabilir yapıdadır — iyi bir pratik.

6.4 Mixer Kod Tekrarı — Değişmedi

⚠️WARN-003 (taşınan) — Ölü Kod

İki ayrı mixer sınıfı hâlâ mevcut: `core/Mixer.h` (Arduino `Servo.h` kütüphanesiyle) ve `core/FixedWingMixer.h` (PIO tabanlı `Output.cpp` ile). `SystemTimer.cpp` yalnızca `FixedWingMixer`'ı kullanıyor; `Mixer.h/.cpp` hiçbir yerden çağrılmıyor (grep ile doğrulandı). Kod tekrarı ve bakım yükü oluşturuyor; `Mixer.h` silinmeli veya açıkça 'legacy/deprecated' olarak işaretlenmelidir.

6.5 Feed-Forward Eksikliği — Değişmedi

Feed-forward kanalı hâlâ yok. Özellikle throttle değişimlerinde oluşan pitch-up/pitch-down eğilimini önceden telafi etmek veya banking (yatış) sırasında yaw koordinasyonu sağlamak için feed-forward terimleri kritik önemdedir. `ArduPilot`'un TECS (Total Energy Control System) sistemi bunu gelişmiş biçimde uygular; `AeroPico`'nun v0.4+ hedeflerine bu eklenmelidir.

6.6 Fixed-Wing Kontrol Özellikleri

Parametre	Değer	Kaynak
Angle P/I/D (roll, pitch)	2.0 / 0.05 / 0.1	<code>config.h</code>
Rate P/I/D (roll, pitch, yaw)	0.1 / 0.01 / 0.01	<code>config.h</code>
Roll açısı limiti	$\pm 30^\circ$	<code>MAX_ROLL_ANGLE</code>
Pitch açısı limiti	$\pm 20^\circ$	<code>MAX_PITCH_ANGLE</code>
Yaw rate limiti	$\pm 100^\circ/\text{s}$	<code>MAX_YAW_RATE</code>
PID çıkış aralığı (rate katmanı)	± 300	<code>core1_entry()</code> çağrı parametreleri
PWM çıkış aralığı	1000-2000 μs , nötr 1500 μs	<code>config.h</code> + <code>Output.cpp</code> clamp

BÖLÜM 7 — Güvenlik Analizi

7.1 Failsafe Mekanizması

RX.cpp'de çift katmanlı failsafe uygulanmış durumda: (1) SBUS protokolünün kendi failsafe biti (sbus_data.failsafe) ve (2) 500 ms'lik yazılımsal zaman aşımı (FAILSAFE_TIMEOUT_MS). Failsafe tetiklendiğinde throttle → PWM_MIN, aileron/elevator/rudder → PWM_NEUTRAL değerlerine çekiliyor ve bu davranış FlightManager::update() içinde rx.isValid() kontrolüyle ikinci kez teyit ediliyor. Bu redundant (yedekli) yapı sağlamdır ve picoport2'de değişmeden korunmuştur.

7.2 Watchdog Timer — Kısmen Çözüldü, Yeni Kör Nokta

✓İlerleme

Önceki raporun en kritik bulgusu (BUG-001: watchdog hiç etkinleştirilmiyor) artık geçerli değil. main.cpp, setup() sonunda watchdog_enable(WATCHDOG_TIMEOUT_MS=2000, true) çağırıyor.

⚠Ancak — Kör Nokta Kalıcı (bkz. §3.7 için tam analiz)

watchdog_update() hem Core 0'daki düşük-öncelikli TelemetryTask'ta hem de Core 1'deki kritik kontrol döngüsünde çağırılıyor. RP2350'nin tek, paylaşılan donanım watchdog'u herhangi bir çekirdekten gelen beslemeye sıfırlandığından, Core 1'in tamamen kilitlenmesi durumunda bile Core 0 watchdog'u beslemeye devam eder ve sistem resetlenmez. Bu, watchdog'un temel amacını (uçuş-kritik döngünün canlılığını garanti etmek) fiilen devre dışı bırakan bir tasarım hatasıdır. Ayrıntılı çözüm önerisi için Bölüm 3.7'ye bakınız.

7.3 Stack Overflow Koruması — Çözüldü

```
extern "C" void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {  
    Serial.printf("[FREERTOS] Stack overflow in %s\n", pcTaskName);    watchdog_update();  
    taskDISABLE_INTERRUPTS(); while (true) { } }
```

✓İlerleme

Önceki raporda eksik olarak işaretlenen vApplicationStackOverflowHook() ve vApplicationMallocFailedHook() artık main.cpp'de tanımlı; platformio.ini derleme bayrakları da configCHECK_FOR_STACK_OVERFLOW=2 ve configUSE_MALLOC_FAILED_HOOK=1 içeriyor. Bu, bellek bozulmasına karşı gerçek bir koruma sağlar — hook tetiklendiğinde sistem kasıtlı olarak kilitlenip watchdog'un devreye girmesini bekliyor (taskDISABLE_INTERRUPTS + while(true)), ki bu noktada watchdog artık etkin olduğu için gerçekten reset atacaktır (bu spesifik senaryoda §7.2'deki kör nokta devre dışıdır, çünkü kilitlenen görev doğrudan watchdog_update() çağrısını da durdurur).

7.4 Heap Yönetimi — Çözüldü

Önceki raporun BUG-003 bulgusu (heap değerinin 182000 olarak hardcode edilmesi) giderilmiştir; main.cpp artık rp2040.getFreeHeap() ile dinamik değeri okuyup BootLogger::printHealthReport()'a iletmektedir.

7.5 Arm/Disarm Güvenliği

ARM şartı: throttle < 1050 VE rudder > 1900, 1 saniye sürdürülmeli. DISARM şartı: throttle < 1050 VE rudder < 1100, 1 saniye sürdürülmeli. Bu, standart RC vericilerinde kullanılan throttle-low + rudder-uç kombinasyonunu taklit ediyor ve kasıtsız arm/disarm'ı önüyor. Mekanizma değişmeden korunmuştur ve sağlamdır.

⚠Küçük Not (taşınan, WARN-009)

updateArmDisarm() içindeki Serial.println() çağrıları hâlâ uçuş döngüsü (SensorTask, 500Hz) bağlamında çalışıyor. Arm/disarm olayları nadir olduğu için pratik etkisi düşüktür, ancak USB seri tamponu doluyorsa Serial.println() bloklayabilir ve döngü gecikmesine yol açabilir.

7.6 Brownout / Safe Mode — Değişmedi

Fren/kaçınma (brownout) algılaması ve güvenli mod mimarisi hâlâ yok. RP2350'nin donanımsal brownout dedektörü mevcut ama yazılımda kullanılmıyor; güç kaybı/düşük voltaj durumunda sistem davranışı tanımsızdır. En azından ADC üzerinden basit bir VBUS/batarya voltaj izleme eklenmesi önerilir (ADC hâlâ tamamen kullanılmıyor, bkz. §9.1).

7.7 Boot Sağlık Raporu — Yeni Kritik Bulgu

main.cpp:setup() içinde, gerçek sensör başlatma sonuçları (Sensors.cpp'nin döndürdüğü _imuAvailable, hasMag, hasBaro değerleri) hiç sorgulanmadan, BootLogger çağrılarına sabit değerler geçiriliyor:

```
BootLogger::okWithValue("MPU6050", "WHOAMI=0x68"); BootLogger::ok("BMP280");
BootLogger::ok("QMC5883L"); BootLogger::ok("RC Receiver"); ...
BootLogger::printHealthReport(500, true, true, true, true, true, false, false,
rp2040.getFreeHeap()); // ^imuOk ^baroOk ^magOk ^rxOk ^dmaOk
– hepsi sabit 'true'
```

❗ BUG-104 — Boot Raporu Sensör Durumunu Yansıtmıyor

SensorManager::init() içinde WHO_AM_I doğrulaması başarısız olsa, mag.begin()/bmp.begin() false dönse veya DMA kanalı alınamasa dahi, main.cpp'deki açılış raporu her zaman 'OK' ve health-report bayraklarını 'true' olarak basmaktadır — çünkü bu çağrılar sensörlerin gerçek dönüş değerlerine değil, sabit sabit-kodlanmış (hardcoded) literallere dayanmaktadır. Bu, güvenlik açısından ciddi bir bulgu olarak değerlendirilmelidir: bir kullanıcı, MPU6050 fiziksel olarak bağlı değilken veya arızalıyken bile seri konsolda 'MPU6050 OK', tüm sağlık bayrakları 'OK' görebilir ve uçağı arm edip uçurmaya çalışabilir. Doğru çözüm, bu çağrılarını SensorManager'ın gerçek durum değişkenlerini (isImuAvailable(), hasMag, hasBaro) okuyacak şekilde yeniden bağlamaktır; bu değerler zaten sınıf içinde mevcuttur ve yalnızca dışa açılması yeterlidir.

BÖLÜM 8 — Haberleşme Analizi

8.1 MAVLink v2 Implementasyonu

MavlinkHandler.cpp, mavlink_parse_char() ile byte-by-byte parsing ve mavlink_msg_*_pack() ile mesaj üretimi yapıyor; PIO UART üzerinden (pio1, GP12/GP13) planlanan ESP32-CAM eşlik eden bilgisayarına (companion computer) gönderiliyor. Deponun Esp32Cam/ klasörü şu an yalnızca bir yer tutucu dosya (boş.txt) içeriyor — ESP32 tarafı firmware'i henüz bu depoda mevcut değil; bu nedenle uçtan uca MAVLink/Blackbox zinciri şu an test edilebilir durumda değildir, yalnızca Pico tarafı hazır.

MAVLink Mesajı	Frekans	Yön	picoport2 Durumu
HEARTBEAT	1 Hz	FC → GCS	Aktif
ATTITUDE	10 Hz	FC → GCS	Aktif
RC_CHANNELS	5 Hz	FC → GCS	Aktif
SYS_STATUS	2 Hz	FC → GCS	Aktif (kısmi — batarya alanları anlamsız, §8.2)
RC_CHANNELS_OVERRIDE	N/A	GCS → FC	ÇÖZÜLDÜ — artık FlightManager'a uçtan uca bağlı
PARAM_REQUEST_LIST / PARAM_SET	N/A	GCS → FC	Derlemede KAPALI + MavlinkHandler'a hiç yönlendirilmiyor (§8.4)
COMMAND_LONG	N/A	GCS → FC	Yok
MISSION_*	N/A	GCS → FC	Yok (planlanan)
GPS_RAW_INT	N/A	FC → GCS	Yok (GPS donanımı henüz entegre değil)

8.2 SYS_STATUS Mesajı — Küçük Bulgu

```
uint16_t battery_remaining = failsafe ? 0 : UINT16_MAX; // MAVLink alanı int8_t (-1..100)!  
... mavlink_msg_sys_status_pack(..., battery_remaining, ...);
```

⚠️WARN-101 — Batarya Alanı Anlamsız Değer Taşıyor

MAVLink'in battery_remaining alanı int8_t tipindedir ve -1 (bilinmiyor) ile 100 (%) arasında bir değer bekler. Kod, uint16_t bir değişkene UINT16_MAX (65535) veya 0 atayıp bunu pack fonksiyonuna geçiriyor; derleyici bunu int8_t'ye daralttığına (truncation) 65535 → -1 (bilinmiyor, kazara doğru), 0 → 0 (%0 dolu, YANLIŞ — batarya izleme hiç yapılmadığı için bu değer 'bilinmiyor' olmalıydı, '%0' değil) sonucunu verir. Batarya izleme donanımı (ADC) hiç bağlı olmadığından, bu alanın her koşulda -1 (MAV_BATTERY_REMAINING sentinel) olarak sabitlenmesi ve failsafe durumunun ayrı bir SYS_STATUS bitiyle (onboard_control_sensors_health) iletilmesi daha doğru olur.

8.3 Blackbox Logger

Blackbox, CSV formatında (BB,timestamp,roll,pitch,yaw,gx,gy,gz,thr,ail,ele,rud,failsafe) veriyi PIO UART üzerinden akıtıyor; TelemetryTask içinde 50Hz'de çağırılıyor. snprintf(buf,128,...) kullanımı stack maliyeti ekliyor ama 2048 word'lük TelemetryTask stack'i için kabul edilebilir. Format basit ama işlevsel; PX4'ün uLog'u veya

Betaflight'ın blackbox protokolüyle kıyaslandığında bant genişliği verimsizdir (CSV metin formatı, binary'ye göre 3-4x daha fazla veri taşır).

8.4 Parametre Sistemi — Devre Dışı ve Bağlantısız

ParamManager, platformio.ini'de yorum satırına alınmış bir makro (;-DMAVLINK_PARAMS_ENABLED ← ESP32 gelince bu satırın başındaki ; kaldır) ile derleme zamanında tamamen devre dışı bırakılmıştır. Ancak makro etkinleştirilse bile ikinci, bağımsız bir sorun devam eder:

❗ BUG-105 — ParamManager Mesaj Yönlendiricisine Hiç Bağlı Değil

MavlinkHandler::_handleMessage() switch-case bloğu yalnızca MAVLINK_MSG_ID_HEARTBEAT ve MAVLINK_MSG_ID_RC_CHANNELS_OVERRIDE case'lerini işliyor; MAVLINK_MSG_ID_PARAM_REQUEST_LIST veya MAVLINK_MSG_ID_PARAM_SET için hiçbir dallanma yok ve paramManager.handleMessage() hiçbir yerden çağrılmıyor (grep ile doğrulanmıştır). Bu demektir ki MAVLINK_PARAMS_ENABLED makrosu açılrsa dahi, bir yer istasyonundan (GCS) gelen PARAM_SET/PARAM_REQUEST_LIST mesajları MavlinkHandler tarafından sessizce yok sayılacak, ParamManager'a asla ulaşmayacaktır. Etkinleştirme için yalnızca makroyu açmak yeterli değildir; MavlinkHandler::_handleMessage() içine paramManager.handleMessage(msg) çağrısının eklenmesi de gerekir.

Ayrıca, önceki raporun WARN-011 bulgusu (ParamManager üzerinden değiştirilen değerlerin canlı PID nesnelerine yansımaması) da geçerliliğini korumaktadır — ancak modül şu an tamamen erişilemez durumda olduğundan bu ikincil bir öncelik hâline gelmiştir.

BÖLÜM 9 — Pico 2 (RP2350) Donanım Analizi

9.1 RP2350 Özelliklerinin Kullanımı

RP2350 Özelliği	Kullanım	Etkinlik
Dual-Core ARM Cortex-M33	Core0: FreeRTOS (2 görev), Core1: bare-metal PID döngüsü	Kullanılıyor — mimari tutarlılık notu §3.1'de
PIO State Machine (2x4 SM)	4x PWM servo + 2x yazılımsal UART = 6/8 SM	Yüksek verimlilik
DMA Controller	MPU6050 I2C RX okuma	Kısmi — TX hâlâ blocking (§4.2)
Donanımsal FPU	Madgwick float işlemleri, PID hesapları	Aktif kullanılıyor
SRAM (~520KB, RP2350)	FreeRTOS heap + uçuş verisi + DMA tamponları	Yeterli, xPortGetFreeHeapSize ile doğrulanabilir
Flash XIP + gerçek XIP cache	Firmware + __not_in_flash_func ile kritik kod RAM'de	İyi kullanılıyor; RP2350'nin cache'i RP2040'a göre bu ihtiyacı zaten kısmen azaltıyor
USB (USBFS)	Seri hata ayıklama (Arduino USB CDC)	Aktif
Donanımsal Watchdog	Etkin (§7.2) ama kör noktalı	Kısmi — mantık hatası mevcut
ADC (12-bit)	Kullanılmıyor	Batarya/voltaj izleme fırsatı değerlendirilmemiş
SPI	Kullanılmıyor	SD kart / harici flash potansiyeli
Donanımsal Zamanlayıcı	pico/time.h ile sleep_us() (Core1 döngü zamanlaması)	Aktif
Brownout Dedektörü	Kullanılmıyor	§7.6'da belirtildiği gibi kritik eksik

9.2 __not_in_flash_func() Kullanımı

Kritik, sık çağrılan fonksiyonlar RAM'de çalışacak şekilde işaretlenmiş: SensorFusion::update()/updateIMU()/computeAngles(), SensorManager::_mpu_parse(), RingBuffer::push()/pop(), PID::compute(). Bu, flash erişim bekleme durumlarını (XIP cache miss) ortadan kaldırarak 500Hz döngüde gerçek performans kazancı sağlar. RP2350'nin RP2040'a göre daha gelişmiş bir XIP önbelleğe sahip olması bu ihtiyacı bir miktar azaltsa da, uygulamanın bu optimizasyonu koruması iyi bir mühendislik pratiğidir.

9.3 Saat (Clock) Yapılandırması

platformio.ini varsayılan saat hızını (133MHz civarı, çekirdek framework varsayılanı) kullanıyor; RP2350 resmi olarak 150MHz'i destekliyor ve topluluk deneyimleri 200-240MHz'e kadar güvenli overclock bildiriyor. Mevcut WCET marjı (§3.6) göz önüne alındığında ek saat hızına şu an gerek yoktur; ancak EKF gibi daha ağır hesaplamalar eklendiğinde (v0.5 hedefi) bir seçenek olarak değerlendirilebilir.

9.4 Bellek Haritası (Statik Analizden Tahmini)

Bölge	Tahmini Boyut	Kullanım
Flash (XIP)	4 MB (board tanımı)	Firmware + kütüphaneler (~700KB-1MB tahmini)
SRAM	~520 KB (RP2350 toplam)	FreeRTOS heap, task stack'leri (2×2048 word), sensör/DMA tamponları
RAM-yerleşik kod (__not_in_flash_func)	Birkaç KB	Fusion, PID, RingBuffer, DMA parse fonksiyonları
Gerçek kullanım	Doğrulanmadı	rp2040.getFreeHeap() runtime'da izlenebilir; statik analiz yerine gerçek ölçüm önerilir

9.5 GPIO / UART Eşlemesi — Kritik Yeni Bulgu

RP2040 ve RP2350'nin GPIO fonksiyon tablosunda (datasheet 'GPIO function select' bölümü), her donanımsal UART çevre biriminin TX/RX sinyalleri yalnızca belirli GPIO pinlerinde kullanılabilir; bu eşleme periyodik olarak (genelde 8 pinlik bloklar hâlinde) tekrar eder ve UART0 ile UART1 farklı pin kümelerine atanmıştır. GP1'in F2 (UART) fonksiyonu, donanımsal olarak yalnızca UART0 RX'tir — UART1 için bu pinde RX fonksiyonu tanımlı değildir.

```
// config.h #define PIN_SBUS_RX 1 // UART0 RX – SBUS alıcı (transistör ile invert) //
drivers/RX.cpp bfs::SbusRx sbus_rx(&Serial2); // Serial2 == UART1 (earlephilhower
çekirdeğinde) void RXManager::init() { // Serial2 → GP1 (RX), transistör ile invert
edilmiş SBUS Serial2.setRX(PIN_SBUS_RX); // GP1'i UART1 RX yapmaya çalışıyor
Serial2.begin(100000, SERIAL_8E2); sbus_rx.Begin(); }
```

❗ BUG-101 — SBUS RX, Fiziksel Olarak Uyumsuz Bir UART'a Bağlanıyor

config.h'daki yorum satırı bile pinin 'UART0 RX' olduğunu doğru şekilde belirtiyor, ancak hemen altındaki RX.cpp bu pini Serial2 (earlephilhower Arduino-Pico çekirdeğinde donanımsal UART1'e karşılık gelir) nesnesine RX pini olarak atıyor. Bu, kodun kendi iç belgelemesiyle çelişen, doğrulanabilir bir tutarsızlıktır. Arduino-Pico çekirdeği setRX() çağrısında istenen pinin ilgili UART çevre birimi için geçerli olup olmadığını kontrol eder; pin geçersizse ya çağrı sessizce yok sayılır ya da UART hiç veri almaz. Sonuç olarak SBUS alıcısının bu haliyle donanımda hiç çalışmaması riski yüksektir. Bu bulgu bir statik kod incelemesinden çıkarılmıştır; gerçek donanımda doğrulanması (SBUS verisinin gerçekten Serial2 üzerinden geldiğinin bir lojik analizör veya en azından RXManager::isValid() durumunun izlenmesiyle teyit edilmesi) şiddetle önerilir. Bulgu doğrulanırsa çözüm, SBUS RX'i ya GP1 ile birlikte Serial1 (UART0) nesnesine, ya da UART1'in geçerli RX pinlerinden birine (örn. GP5 — ki bu I2C SCL ile çakışır — veya GP9, GP21) taşımaktır.

Bu bulgunun önemi, projenin uçuş güvenliği zincirinin en temel girdisini (pilotun RC komutları) etkilemesinden kaynaklanmaktadır: RX.cpp zaten sağlam bir failsafe mantığına sahiptir (§7.1), ancak SBUS verisi donanımsal olarak hiç ulaşmıyorsa sistem sürekli failsafe modunda kalacak (ki bu görece güvenli bir başarısızlık biçimidir) ya da — daha endişe verici bir ihtimalle — pin çakışması UART0'ı kullanan başka bir çevre birimini (örn. USB-seri köprüsü, varsa) bozabilir. Bu nedenle rapor, bu maddeyi Bölüm 1.4'te 'en yüksek öncelik' olarak sınıflandırmıştır.

BÖLÜM 10 — PX4 / ArduPilot Karşılaştırması

10.1 Özellik Karşılaştırma Tablosu

Özellik	AeroPico picoport2	PX4 v1.15	ArduPilot 4.5
HAL Katmanı	△	✓	✓
FreeRTOS Scheduler	△	✓	✓
Dual-Core Kullanımı	✓	△	△
PIO / Özel Donanım Hızlandırma	✓	✗	✗
Cascaded PID	✓	✓	✓
Anti-Windup	✓	✓	✓
EKF / EKF2	✗	✓	✓
Sensor Fusion (9-DOF)	✓	✓	✓
Feed-Forward	✗	✓	✓
Uçuş Modları (state machine)	✗	✓	✓
Mission / Waypoint	✗	✓	✓
GPS Navigasyon	✗	✓	✓
MAVLink	✓	✓	✓
Parametre Sistemi (çalışır durumda)	✗	✓	✓
Kalıcı Parametre Depolama	✗	✓	✓
Watchdog (gerçekten etkili)	△	✓	✓
Stack Guard	✓	✓	✓
Batarya İzleme	✗	✓	✓
Sensör Kalibrasyonu	✗	✓	✓
Hard/Soft-Iron Kompanzasyonu	✗	✓	✓
TECS (Enerji Kontrolü)	✗	✓	✓
L1 Navigasyon Kontrolcüsü	✗	✓	✓
Çoklu Platform Desteği	✗	✓	✓
Kod Okunabilirliği (yeni katılımcı için)	✓	△	△
Başlangıç Kolaylığı (kurulum)	✓	✗	✗
RP2040/2350'ye Özel Optimizasyon	✓	✗	✗
Otomatik Test Altyapısı	✗	✓	✓

10.2 AeroPico'nun Üstün Olduğu Alanlar

- RP2350'ye özgü PIO ve DMA optimizasyonu — PX4/ArduPilot'un genel amaçlı HAL katmanı bu derece düşük seviyeye inmiyor.
- Kod okunabilirliği ve eğitim değeri — 2.415 satırlık kod tabanı, tek oturuşta baştan sona okunup anlaşılabilir; PX4/ArduPilot yüz binlerce satırlık, çok katmanlı sistemlerdir.
- Çift çekirdek kullanımı — PX4/ArduPilot çoğunlukla tek çekirdek üzerinde work-queue/scheduler tabanlı çalışır; AeroPico donanımsal çekirdek ayırımını doğrudan kullanıyor.
- Minimum bağımlılık — Adafruit sürücüler ve SBUS kütüphanesi dışında harici bağımlılık az.
- Başlangıç eşiği düşük — PlatformIO + Pico kurulumu, PX4/ArduPilot'un çok adımlı SDK/toolchain kurulumuna göre çok daha erişilebilir.

10.3 Geliştirilmesi Gereken Alanlar

- EKF/EKF2 entegrasyonu: GPS + barometre + IMU füzyonu için Extended Kalman Filter — şu an tamamen yok.
- Uçuş modu mimarisi: MANUAL, FBWA, CRUISE, AUTO — durum makinesi (state machine) ile yönetim.
- HAL genelleştirmesi: Farklı sensör/platform desteği için soyutlama katmanı.
- Kalıcı parametre depolama: Flash API veya littlefs entegrasyonu (ön koşul: ParamManager'ın önce çalışır hâle getirilmesi, §8.4).
- Sensör kalibrasyon rutini: Boot sırasında gyro/accel bias hesaplama.
- Otomatik test: Host-side birim testleri (PID, SensorFusion, Mixer) hiç yok; PX4/ArduPilot'un olgunluğunun büyük kısmı bu altyapıdan gelir.

BÖLÜM 11 — Kod Denetimi (Bug Hunt) — Konsolide Liste

Bu bölüm, önceki bölümlerde tespit edilen tüm bulguları tek bir önceliklendirilmiş listede toplar. Kimlik numaraları (BUG-1xx / WARN-1xx) picoport2 dalına özgü yeni bulguları; metinde '(taşınan)' notu geçen maddeler ise Haziran 2026 tarihli önceki incelemeden hâlâ geçerliliğini koruyan bulguları belirtir.

11.1 Kritik Hatalar (Uçuş Güvenliğini Doğrudan Etkileyebilir)

BUG-101 — SBUS RX, GPIO1'de Desteklenmeyen Bir UART'a Bağlanıyor

Konum: src/drivers/RX.cpp, src/config.h. Serial2 (UART1) nesnesi, yalnızca UART0 RX fonksiyonunu destekleyen GP1'e bağlanıyor. Ayrıntılı analiz: Bölüm 9.5. Öncelik: ACİL — donanımda doğrulanmalı.

BUG-102 — Watchdog, Core 1 Kilitlenmesini Tespit Edemiyor

Konum: main.cpp (taskTelemetry), core/SystemTimer.cpp (core1_entry). Tek paylaşılan donanım watchdog'u, düşük öncelikli Core 0 görevinden gelen düzenli besleme sayesinde Core 1'in tamamen kilitlenmesi durumunda dahi sıfırlanmaya devam eder. Ayrıntılı analiz: Bölüm 3.7 / 7.2. Öncelik: KRİTİK.

BUG-103 — Boot Sağlık Raporu Sabit-Doğru Değerler Basıyor

Konum: main.cpp:setup(). BootLogger::ok()/printHealthReport() çağrıları, SensorManager'ın gerçek _imuAvailable/hasMag/hasBaro durumlarını sorgulamadan sabit 'true' geçiriyor. Ayrıntılı analiz: Bölüm 7.7. Öncelik: KRİTİK — yanlış güvenlik hissi riski.

11.2 Orta Seviye Sorunlar

WARN-101 — RingBuffer SPSC Varsayımı İhlal Ediliyor (Çoklu Tüketici)

Konum: core/RingBuffer.h kullanım şekli — main.cpp (taskTelemetry) ve core/SystemTimer.cpp (core1_entry) aynı ring buffer'ı bağımsız olarak tüketiyor. Ayrıntılı analiz: Bölüm 3.4. Öncelik: ORTA-YÜKSEK.

WARN-102 — asin() Argümanı Sınırlanmıyor (Gimbal Lock Bölgesinde NaN Riski)

Konum: core/SensorFusion.cpp:computeAngles(). Pitch $\pm 90^\circ$ 'ye yaklaşırken float yuvarlama hatası asin() argümanını [-1,1] dışına taşıyabilir. Ayrıntılı analiz: Bölüm 5.4. Öncelik: ORTA — düşük olasılık, yüksek etki, tek satırlık düzeltme.

WARN-103 — ParamManager Mesaj Yönlendiricisine Bağlı Değil

Konum: telemetry/MavlinkHandler.cpp:_handleMessage(). PARAM_REQUEST_LIST/PARAM_SET case'leri yok; paramManager.handleMessage() hiç çağrılmıyor. Ayrıntılı analiz: Bölüm 8.4. Öncelik: ORTA (modül zaten derlemede kapalı).

WARN-104 — Sensör İsimleri Boot Logunda Yanlış Raporlanıyor

Konum: main.cpp:setup(). 'BMP280' ve 'QMC5883L' basılıyor; gerçek sürücüler BMP085 ve HMC5883L. Ayrıntılı analiz: Bölüm 4.3. Öncelik: ORTA — hata ayıklamada yanıltıcı.

WARN-105 — SYS_STATUS Batarya Alanı Anlamsız Değer Taşıyor

Konum: telemetry/MavlinkHandler.cpp:sendSysStatus(). uint16_t → int8_t daralmasında %0 dolu ile 'bilinmiyor' karışıyor. Ayrıntılı analiz: Bölüm 8.2. Öncelik: DÜŞÜK-ORTA.

WARN-003 (taşınan) — Mixer.h Kullanılmıyor

Konum: src/core/Mixer.cpp/h. FixedWingMixer ile aynı işlevi yapan, hiçbir yerden çağrılmayan ikinci bir mixer sınıfı. Öncelik: ORTA — bakım yükü.

WARN-004 (taşınan) — IMU.cpp Kullanılmıyor

Konum: src/drivers/IMU.cpp/h. Legacy complementary filter; SensorFusion (Madgwick) ile değiştirilmiş ama silinmemiş. Öncelik: ORTA — bakım yükü.

WARN-001 (taşınan) — _mpu_start_dma_read() İçinde Blocking I2C Yazımı

Konum: src/drivers/Sensors.cpp. Register adresi yazımı hâlâ i2c_write_blocking() ile yapılıyor; tam non-blocking pipeline için DMA'ya alınmalı. Öncelik: ORTA.

WARN-002 (taşınan) — Wire.h Üzerinden Blocking Mag/Baro Okuması

Konum: src/drivers/Sensors.cpp:update(). mag.getEvent()/bmp.getEvent() en kötü senaryoda birkaç ms gecikme ekleyebilir. Öncelik: ORTA.

11.3 Düşük Öncelikli Uyarılar

- MathUtils.cpp/h tanımlı ama hiçbir yerden çağrılmıyor — ölü kod.
- types.h içindeki RC_Values struct'ı hiçbir yerde kullanılmıyor — ölü kod (taşınan, WARN-012).
- Serial.h ve Serial.cpp tamamen boş dosyalar — yer tutucu kalmış (taşınan, WARN-013).
- telemetry/SerialCom.h/.cpp yalnızca TODO yorumu içeriyor — planlanmış ama başlanmamış.
- config.h'daki LOOP_TIME_MS (=20) sabiti hiçbir yerden kullanılmıyor; gerçek döngü periyodu def.h'daki farklı isimli LOOP_TIME (=2000µs) sabitinden geliyor — iki benzer isimli, farklı anlamlı sabit karışıklığa açık.
- FlightManager::getRoll()/getPitch()/getYaw()/getGyroX() vb. metotların her biri kendi başına _getLatest() çağırarak ring buffer'ı bağımsız tüketiyor (taşınan, WARN-008) — tek bir snapshot metodu yeterli olurdu.
- updateArmDisarm() içindeki Serial.println() çağrıları uçuş döngüsü bağlamında çalışıyor (taşınan, WARN-009).
- Esp32Cam/ klasörü yalnızca boş bir yer tutucu dosya (boş.txt) içeriyor; Blackbox/MAVLink zincirinin alıcı ucu (ESP32-CAM firmware'i) bu depoda henüz mevcut değil.
- Ground Control System/ klasöründeki arayüz görseli (UI-UX.png) bir tasarım hedefini gösteriyor; buna karşılık gelen bir GCS yazılımı depoda bulunmuyor.

BÖLÜM 12 — Geliştirme Yol Haritası

Aşağıdaki yol haritası, önceki incelemenin v0.3→v2.0 planını temel alır ancak picoport2'de tespit edilen yeni kritik bulguları (§1.4, §11.1) en üst önceliğe taşıyacak şekilde güncellenmiştir.

12.1 v0.3 — Acil Düzeltmeler (1-2 hafta)

- SBUS RX pin/UART eşlemesini donanımda doğrulama ve düzeltme (BUG-101) — KRİTİK, ilk yapılacak iş.
- Watchdog kör noktasının giderilmesi: Core 1 canlılık damgası + Core 0 tarafından izleme, veya watchdog_update()'in yalnızca Core 1'den çağırılması (BUG-102).
- Boot sağlık raporunun gerçek sensör durumlarını okuyacak şekilde düzeltilmesi (BUG-103).
- RingBuffer'in gerçek SPSC kullanımına döndürülmesi: tek tüketici + peek() tabanlı ikincil okuma (WARN-101).
- asin() argümanının constrain() ile sınırlanması (WARN-102) — tek satırlık, düşük riskli.
- Sensör isimlerinin boot logunda düzeltilmesi (WARN-104).

12.2 v0.4 — Temizlik ve Sağlama (2-4 hafta)

- Ölü kod temizliği: Mixer.h/.cpp, IMU.cpp/h, MathUtils.cpp/h, types.h RC_Values, Serial.h/.cpp.
- ParamManager'ın MavlinkHandler mesaj yönlendiricisine bağlanması ve PID nesnelerine canlı parametre uygulaması (WARN-103, taşınan WARN-011).
- Gyro/accel boot kalibrasyonu: ilk N örneğin ortalaması ile bias hesaplama.
- SYS_STATUS batarya alanının MAV_BATTERY_REMAINING sentinel'iyle düzeltilmesi (WARN-105).
- _mpu_start_dma_read() içindeki I2C TX'in DMA'ya alınması (WARN-001, taşınan).

12.3 v0.5 — Özellik Genişletme (1-2 ay)

- Uçuş modu durum makinesi: MANUAL, FBWA (Fly-By-Wire-A), STABILIZE.
- GPS NMEA parser entegrasyonu (UART1, GP8/GP9 — bu port için de aynı GPIO/UART doğrulaması BUG-101 metodolojisiyle yapılmalı).
- Barometrik irtifa tutma (altitude hold) — basit P kontrolcüsü.
- Kalıcı parametre depolama: RP2350 flash API veya littlefs.
- Manyetometre hard-iron kalibrasyonu (min/max yöntemi).
- Binary Blackbox formatı (CSV yerine) — bant genişliği optimizasyonu.
- ADC tabanlı batarya voltaj izleme + brownout güvenli modu.

12.4 v1.0 — Kararlı Sürüm (3-5 ay)

- Host-side birim testleri: PID, SensorFusion, Mixer, RingBuffer (özellikle çoklu-tüketici senaryosu için regresyon testi).
- Hardware-in-the-loop (HIL) simülasyon desteği.
- EKF veya EKF2 entegrasyonu: GPS + Baro + IMU füzyonu.
- Kapsamlı Doxygen API dokümantasyonu.
- QGroundControl / Mission Planner ile tam MAVLink dialect uyumu.

12.5 v2.0 — Platform Genişlemesi

- HAL soyutlaması: farklı Pico/RP2350 kartları ve sensör kombinasyonları için genelleştirme.
- L1 navigasyon kontrolcüsü ve TECS entegrasyonu ile AUTO uçuş modu.
- Quadrotor mixer desteği araştırması.
- SITL (Software-in-the-loop) desteği.

EKLER

Ek A — Görev / Zamanlama Özeti

Görev / Döngü	Çekirdek	Öncelik	Periyot	Kritiklik
SensorTask	Core 0 (FreeRTOS)	2	2 ms (500 Hz)	Yüksek — sensör + fusion + RX
TelemetryTask	Core 0 (FreeRTOS)	1	20 ms (50 Hz)	Orta — MAVLink + Blackbox + watchdog besleme
core1_entry (PID+Mixer)	Core 1 (bare-metal)	N/A	2 ms (500 Hz)	Kritik — uçuş kontrolü, ama watchdog'la yeterince korunmuyor (§3.7)

Ek B — Risk Matrisi

Risk	Olasılık	Etki	Azaltma Stratejisi
SBUS RX GPIO/UART uyumsuzluğu (BUG-101)	Orta-Yüksek (doğrulama gerekli)	Kritik	Pin/UART eşlemesini donanımda test et, gerekirse GP1→UART0 veya SBUS'ı geçerli bir UART1 pinine taşı
Watchdog kör noktası (BUG-102)	Düşük (yalnızca Core1 kilitlenirse)	Kritik	Core1 canlılık damgası + Core0 tarafından bağımsız izleme
Boot raporu yanlış 'OK' basıyor (BUG-103)	Yüksek (her açılışta)	Yüksek	BootLogger çağrılarını gerçek sensör durumlarına bağla
RingBuffer çoklu tüketici yarışı (WARN-101)	Orta	Orta	Tek tüketici + peek() tabanlı ikincil erişim
asin() domain hatası / NaN yayılımı (WARN-102)	Düşük	Orta-Yüksek	constrain(x,-1,1) eklemek
Sensör kalibrasyonsuz → hatalı açılar	Yüksek	Yüksek	Boot kalibrasyon rutini
Wire.h blocking I2C → döngü jitter'ı	Orta	Orta	Native SDK ile DMA/async yeniden yazım
Param reset sonrası kayboluyor / hiç çalışmıyor	Yüksek	Orta	Mesaj yönlendirme + flash kalıcı depolama
Brownout / güç kaybı davranışı tanımsız	Düşük-Orta	Yüksek	ADC VBUS izleme + güvenli mod

Ek C — Modül Puan Özeti

Modül / Kriter	Puan	Not
Mimari Genel	8/10	Katmanlı yapı korunmuş, çekirdek kullanımı net
PIO / Donanım Kullanımı	9/10	RP2350 kapasitesi verimli kullanılıyor
DMA Sensör Okuma	7/10	TX hâlâ blocking, kısmi DMA

Modül / Kriter	Puan	Not
Madgwick (9-DOF)	8/10	Tam gradyan-inişi implementasyonu doğru
Madgwick/Mahony (6-DOF)	7/10	İvme düzeltmesi eklendi (İYİLEŞTİ), ama 9-DOF'tan farklı algoritma ailesi
Cascaded PID + Anti-Windup	8/10	Koşullu entegrasyon anti-windup eklendi (İYİLEŞTİ)
FixedWingMixer	8/10	Gain+trim+constrain, temiz; Mixer.h ile kod tekrarı var
RXManager / SBUS	4/10	Failsafe mantığı mükemmel ama GPIO/UART uyumsuzluğu riski tüm modülü geçersiz kılabilir
MAVLink Handler	8/10	RC override artık uçtan uca, callback mimarisiyle ayrıştırılmış
Blackbox	7/10	İşlevsel ama alıcı taraf (ESP32) depoda yok, binary format yok
ParamManager	2/10	Derlemede kapalı ve mesaj yönlendiricisine hiç bağlı değil
Güvenlik / Watchdog	5/10	Etkin ama kör noktalı; stack/heap koruması tam
Kod Kalitesi / Okunurluk	8/10	Yorum zengin, modüler; 4 ölü modül birikmiş
Test / Doğrulama	2/10	Test altyapısı hâlâ yok

Ek D — Önceliklendirilmiş TODO Listesi

Öncelik	Görev	Dosya	Tahmini Süre
❗ ACİL	SBUS GPIO/UART eşlemesini donanımda doğrula ve düzelt	RX.cpp, config.h	1-3 saat + donanım testi
❗ KRİTİK	Watchdog'u Core1 canlılığına bağla	main.cpp, SystemTimer.cpp	2-3 saat
❗ KRİTİK	Boot sağlık raporunu gerçek sensör durumuna bağla	main.cpp, Sensors.h	1 saat
❗ YÜKSEK	RingBuffer'lı tek-tüketici kullanımına döndür	FlightManager.cpp, main.cpp	2 saat
❗ YÜKSEK	asin() argümanını sınırla	SensorFusion.cpp	15 dk
❗ YÜKSEK	Gyro/accel boot kalibrasyonu ekle	Sensors.cpp	3 saat
❗ ORTA	ParamManager'ı mesaj yönlendiricisine bağla + PID'e uygula	MavlinkHandler.cpp, ParamManager.cpp	3 saat
❗ ORTA	Ölü kod temizliği (Mixer, IMU, MathUtils, RC_Values)	core/, drivers/, utils/, types.h	1-2 saat
❗ ORTA	Boot log sensör isimlerini düzelt	main.cpp	15 dk
❗ ORTA	SYS_STATUS batarya alanını sentinel ile düzelt	MavlinkHandler.cpp	20 dk
❗ DÜŞÜK	I2C TX'i DMA'ya al (tam non-blocking)	Sensors.cpp	4 saat
❗ DÜŞÜK	Binary blackbox formatı	Blackbox.cpp	4 saat
❗ DÜŞÜK	Mag hard-iron kalibrasyonu	Sensors.cpp	3 saat

Kapanış Notu

AeroPico FC picoport2, önceki incelemenin en kritik güvenlik bulgularından üçünü (watchdog devre dışı, stack overflow koruması yok, heap raporlaması sahte) somut biçimde gidermiş; MAVLink bağımlılık mimarisini ve PID anti-windup'ını önerilen yönde iyileştirmiştir. Bu, gerçek bir mühendislik ilerlemesidir. Aynı zamanda, RP2350 portu sürecinde ortaya çıkan ve daha önce belgelenmemiş üç yeni bulgu (SBUS GPIO/UART uyumsuzluğu, watchdog'un Core1'e özgü kör noktası, boot sağlık raporunun güvenilmezliği), projenin bir sonraki geliştirme turunda ele alınması gereken en yüksek öncelikli maddeler olarak öne çıkmaktadır. Bu üç madde giderildiğinde proje, iddia ettiği güvenlik seviyesine fiilen ulaşmış olacaktır.